

CoLoMoTo_Tutorial_Sizek_2025

May 15, 2025

1 Reproducible Boolean model analyses and simulations with the CoLoMoTo software suite - a tutorial

Vincent Noël¹, Aurélien Naldi², Laurence Calzone^{1,#}, Loic Paulevé^{3,#}, Denis Thieffry^{1,2,#}

¹ Institut Curie, INSERM U900, Mines Paris, PSL Université, Paris, France.

² Département de biologie, PSL Université, Paris, France.

³ LaBRI, CNRS UMR 5800, INP, Université de Bordeaux, Talence, France.

These authors contributed equally to the study.

January 22nd, 2025.

1.1 Abstract

This tutorial provides stepwise instructions to install the 20 tools integrated in the CoLoMoTo software suite, to develop reproducible dynamical analyses of logical models of complex biological molecular networks.

The tutorial specifically focuses on the analysis of a previously published model of the regulatory network controlling mammalian cell proliferation. It includes chunks of python code to reproduce several of the results and figures published in the original article and further extend these results with the help of a selection of tools included in the CoLoMoTo suite. The notebook covers the visualisation of the network with the tool GINsim, an attractor analysis with bioLQM, the computation of synchronous attractors with BNS, the extraction of modules from the full model, MaBoSS simulations of the wild-type model, as well as of selected mutants, and finally the delineation of compressed probabilistic state transition graphs.

The integration of all these analyses in an executable Jupyter notebook greatly eases their reproducibility, as well as the inclusion of further extensions. This notebook can further be used as a template and enriched with other CoLoMoTo tools to enable comprehensive dynamical analyses of biological network models.

1.2 1. Introduction

Since the seminal studies from Motoyosi Sugita [1], Stuart Kauffman [2] and René Thomas [3], Boolean models have been increasingly applied to biological signalling and regulatory networks [4, 5, 6, 7). With the rise of high-throughput functional genomic methods and the development of molecular pathway knowledge databases, modellers are now coping with networks encompassing hundreds of regulatory components controlling cell fate decisions for normal and pathological conditions (see e.g. [8]).

In parallel, dozens of computational tools have been developed to infer, edit and analyse the dynamical properties of Boolean networks, using different programming languages and model formats (see e.g. [9] and references therein). However, like experimental biologists, modellers are facing serious reproducibility challenges, which have been stressed in recent years (see[10], focusing on modelling studies using ordinary differential equations).

About a decade ago, several international teams working on the development of Boolean modelling tools gathered efforts to address these challenges. The aim was to delineate a common standard format to foster the exchange and reuse of qualitative models between different software tools, which took the form of a dedicated extension of a new, modular release of the popular SBML format, known as sbml-qual [11].

The second effort was put on the integration of a growing set of tools in a *Docker* container to ease their installation and articulation in flexible analysis workflows. The first release of the resulting *Common Logical Modelling Toolbox* (or *CoLoMoTo*) encompassed six complementary software tools [12]. Since then, the number of tools integrated into the *CoLoMoTo* suite has been constantly increasing, reaching a total of twenty in 2024. The tools currently integrated in the suite are listed in **Figure 1**, with key characteristics.

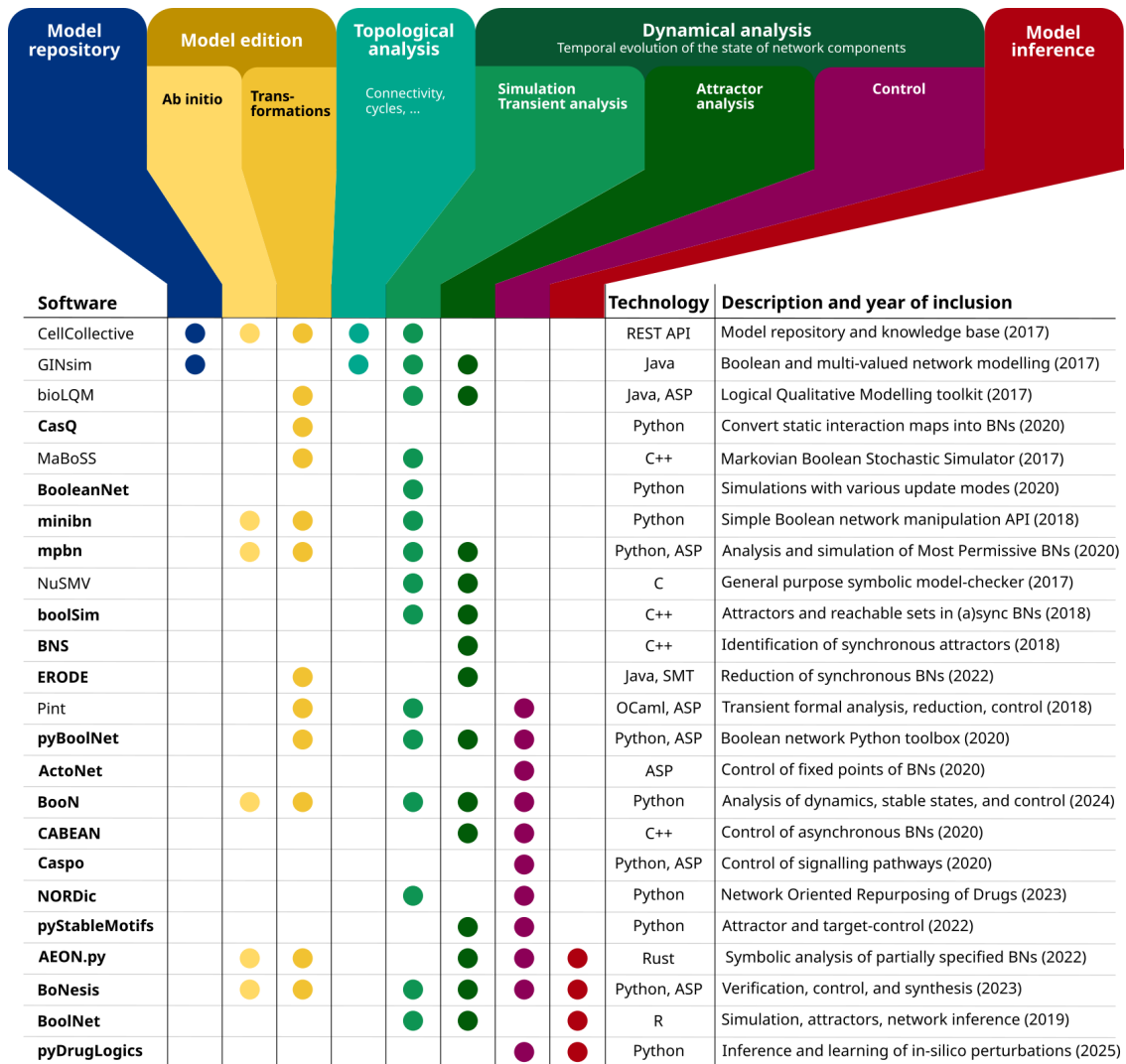


Figure 1. Tools integrated in the CoLoMoTo suite. Tools with names in bold have been added after the 2018 publication.

This tutorial aims to help modellers take advantage of the multiple tools integrated into the *CoLoMoTo* software suite and perform sophisticated Boolean model analyses of complex signalling/regulatory networks. This tutorial relies on a comprehensive model of key signalling pathways and a core regulatory network controlling mammalian cell proliferation, published by Sizek et al. in 2019 [13]. The proposed tasks performed with selected tools aim at comparing the model outcomes to expected experimental behaviours (e.g., existence of cell states in response to receptor activation, cell differentiation, etc.), predicting the results of potential perturbations (such as mutations in patients), or anticipating the effect of treatments. As we shall see, the tutorial includes chunks of *python* code to reproduce several of the results and figures published in the original article and further extends these results with the help of several tools included in the *CoLoMoTo* software suite. This tutorial specifically aims to demonstrate the added value of combining different tools to perform reproducible dynamic analyses, and shows the effectiveness of using python to orchestrate these different analyses.

1.3 2. Material and methods

1.4 2.1. Installation

This notebook can be run on a recent personal computer with at least 16 Go of RAM and about 10 Go of available disk space. The *CoLoMoTo* environment can be installed either as a *Docker* container or as individual *conda* packages.

1.4.1 2.1.1. Installation of the CoLoMoTo Docker container

Docker allows packaging a complete pre-installed software environment, so that applications can run consistently across different machines and operating systems. As such, it is well adapted to reproduce complex computational workflows, relying on numerous software. *Docker* and *Python* need to be pre-installed, go to your folder and then type in a terminal:

```
pip install -U colomoto-docker
colomoto-docker -V 2025-03-01 --bind .
```

1.4.2 2.1.2. Installation of CoLoMoTo conda packages

As an alternative to using *Docker*, one can rely on *conda* to create a software environment able to reproduce the notebook on *Linux* and *macOS* computers by typing the following commands in a terminal:

```
conda create -n sizek19 -c colomoto -c potassco -c conda-forge ginsim-python bns-python boolsim
```

To launch this notebook, type the command:

```
jupyter notebook
```

Because *BNS* and *boolSim* executables are not available for *Windows®* operating systems, the complete notebook can be executed only within *Docker* on these systems.

More generally, as the versions of package dependencies cannot be fully controlled with *conda*, reproducibility is better ensured by using the *Docker container*.

1.5 2.2. Model

The model published by Sizek et al. [13] has been imported and edited with the *GINsim* software [14]. It is publicly available in the GINsim *zginml* model format (including the layout, to be open with *GINsim*, version 3.0), as well as in the *sbml-qual* format, in the *GINsim* model repository (<http://ginsim.org/>). Hence, the model can be directly loaded from the notebook using the link to the corresponding *GINsim* model repository entry.

Alternatively, the model can be downloaded locally and the loading command modified accordingly, thereby enabling to work offline. The model was initially conceived as the combination of six signalling/regulatory modules contributing to the decision between cell proliferation versus death. A first module accounts for the orchestration of the cell cycle, resulting in the successive activation of the main markers of the cell division process, the cyclins. A second module accounts for the control of apoptosis, represented by the activation of Casp3 and the influence of the TRAIL death signal. A third module represents the regulation of growth via the MAPK and PI3k/AKT pathways and their activation by environmental factors. A fourth module represents the G1 restriction point. A fifth module represents the regulation of the origin of replication. The last module accounts for DNA damage. Connecting all these modules (displayed with different colours in Figure 2), the model can account for many different cellular situations, including genetic alterations common in cancer, and can successfully reproduce observed phenotypes.

1.6 3. Model analysis

The following sections of notebook cover the following analyses:

- Loading of the packages required for the tutorial
- Loading of the model and visualisation of the network with *GINsim*.
- Attractor analysis with *bioLQM*.
- Computation of synchronous attractors with *BNS*.
- Analysis of the attractors of the model modules using *bioLQM*, *BNS* and *boolSim*.
- Stochastic simulations of the wild type model with *MaBoSS*.
- Stochastic dynamical simulations of mutants with *MaBoSS*.
- The construction of compressed probabilistic transition graphs with *MaBoSS*.

This chaining of steps constitutes a coherent analysis workflow, starting with the identification of attractors (stable states or cycles), considering different updating schemes, before performing more precise and complex stochastic simulations, for the wildtype case, as well as for selected mutant backgrounds. Each step is performed using one selected software tool, but other CoLoMoTo tools could be used to perform similar analyses (cf. Figure 1).

1.6.1 3.1. Loading required packages

Before running the analyses, the packages to be used need to be loaded. For this tutorial, only the necessary CoLoMoTo packages are selected. The other packages listed in Figure 1 can be loaded according to the instructions provided at <https://colomoto.github.io/colomoto-docker/>.

Note that these packages can also be loaded later on, just before their use.

Pieces of code will be shown in this tutorial in boxes as follows. They represent code cells extracted from the python jupyter notebook.

```
[1]: %%time
import numpy as np
import pandas as pd
import seaborn as sns
import ginsim
import biolqm
import maboss
import bns
import boolsim
from colomoto_jupyter import tabulate # for fixpoint table display
import matplotlib.pyplot as plt # for modifying plots
import os
```

This notebook has been executed using the docker image `colomoto/colomoto-docker:2025-03-01`

CPU times: user 2.89 s, sys: 248 ms, total: 3.13 s

Wall time: 2.19 s

1.6.2 3.2 Loading and Visualisation of the model with GINsim

bioLQM [15] was used to import the model published by Sizek and colleagues, after updating its layout with *GINsim*. The resulting model has the same logical rules as the original one and can be found in the *GINsim* repository at the address: <http://ginsim.org/node/258>

To load the model and visualise its regulatory graph, we run the following code cell:

```
[2]: # Loading and visualisation of the model
model_ginsim = ginsim.load("http://ginsim.org/sites/default/files/
↳CellCycleControl_Sizek_PCB_2019.zginml")
ginsim.show(model_ginsim)
```

/notebook/CellCycleControl_Sizek_PCB_2019.zginml

```
[2]: <IPython.core.display.HTML object>
```

This regulatory graph encompasses 87 components representing different molecular species and cellular processes involved in the regulation of cell proliferation and apoptosis in a generic human cell. The authors conceived this complex model as an association of six main functional modules:

- The growth signaling module (green nodes) incorporates growth signaling pathways driving cell cycle commitment, responsible for modeling the dynamics of PI3K, AKT1, MAPK, and mTORC.
- The restriction switch module (blue nodes): is responsible for commitment to DNA synthesis.
- The origin licensing switch module (yellow nodes) controls licensing and firing of replication origins.
- The phase switch module (pink nodes) controls cell cycle progression (G2 -> M -> G1), taking into account the role of Polo-like kinase 1 (Plk1) during mitosis.
- The apoptosis switch module (red nodes) implements the decision between cell survival versus apoptosis.
- The cellular processes module (orange nodes) includes nodes helping to identify the phenotypic state and covering additional regulatory mechanisms.

In the regulatory graph, green arrows denote activatory interactions, while red T arcs represent inhibitions. As many model components are regulated by multiple components, their behaviour in response to regulatory input levels is encoded in Boolean rules, which combine literals (i.e., node

ids) with the classical Boolean operators NOT (written “!” in *GINsim*), OR (“|”), and AND (“&”). The rules defined in the initial publication were encoded in the *zginml* file available in the *GINsim* model repository.

1.6.3 3.3. Attractor analysis with *bioLQM*

Attractors represent the long-term (asymptotic) behaviours of the model. In the case of Boolean models, attractors can correspond to stable states (with all components frozen at level 0 or 1) or to periodic/cyclic behaviour (with a subset of components switching between values 0 or 1). Attractors are often associated with cellular phenotypes, cell fates (e.g. cell proliferation or cell death), or cellular types in the case of cell differentiation models. Their analysis allows the modeller to verify that the model reproduces the dynamical properties expected for the biological system in normal or perturbed conditions.

To obtain a first overview of the asymptotic properties of the model, we can use the software tool *bioLQM* [15], which includes an efficient algorithm to identify all stable states, regardless of their reachability [16].

We first need to convert our model in the *biolqm* format.

```
[3]: # conversion of the model for biolqm
model_biolqm = ginsim.to_biolqm(model_ginsim)
```

Using *bioLQM*, the stable states of the model can be computed in the following code cell. They correspond to long-term solutions of the model.

```
[4]: %%time
# Compute fixpoints
fps = biolqm.fixpoints(model_biolqm)
print(len(fps), "fixpoints") # shows the number of fixpoints
tabulate(fps) # table displaying all the fixpoints
```

8 fixpoints

CPU times: user 123 ms, sys: 66.5 ms, total: 190 ms

Wall time: 392 ms

```
[4]:
```

	GF	RTK	Grb2	Ras	RAF	mTORC2	PI3K	PIP3	PDK1	AKT_B	p110_H	PI3K_H	\
0	0	0	0	0	0	1	0	0	0	0	1	0	
1	0	0	0	0	0	1	0	0	0	0	1	0	
2	1	0	0	0	0	1	0	0	0	0	1	0	
3	1	0	0	0	0	1	0	0	0	0	1	0	
4	1	0	0	0	0	1	0	0	0	0	1	0	
5	1	0	0	0	0	1	0	0	0	0	1	0	
6	1	1	0	0	0	1	1	1	1	1	1	0	
7	1	1	0	0	0	1	1	1	1	1	1	0	

	AKT_H	FoxO3	PLCgamma	Neddl4	FoxO1	p21_mRNA	TSC2	PRAS40	Rheb	\
0	0	1	0	0	1	1	1	1	0	
1	0	1	0	0	1	1	1	1	0	
2	0	1	0	0	1	1	1	1	0	

3	0	1	0	0	1	1	1	1	0
4	0	1	0	0	1	1	1	1	0
5	0	1	0	0	1	1	1	1	0
6	0	1	0	0	1	1	1	1	0
7	0	1	0	0	1	1	1	1	0

	mTORC1	S6K	eIF4E	GSK3	p21	pRB	p27Kip1	Myc	CyclinD1	E2F1	CyclinE	\
0	0	0	0	1	0	0	0	0	0	0	0	
1	0	0	0	1	0	0	0	0	0	0	0	
2	0	0	0	1	0	0	0	0	0	0	0	
3	0	0	0	1	0	0	0	0	0	0	0	
4	0	0	0	1	0	0	0	0	0	0	0	
5	0	0	0	1	0	0	0	0	0	0	0	
6	0	0	0	1	1	1	1	0	0	0	0	
7	0	0	0	1	1	1	1	0	0	0	0	

	ORC	Cdc6	Cdt1	Pre_RC	geminin	CyclinA_mRNA	Emi1	FoxM1	Cdc25A	\
0	0	0	0	0	0	0	1	0	0	
1	0	0	0	0	0	0	1	0	0	
2	0	0	0	0	0	0	1	0	0	
3	0	0	0	0	0	0	1	0	0	
4	0	0	0	0	0	0	1	0	0	
5	0	0	0	0	0	0	1	0	0	
6	0	0	0	0	0	0	0	0	0	
7	0	0	0	0	0	0	0	0	0	

	CyclinA	Wee1	UbcH10	CyclinB	Cdc25B	Plk1	Cdc25C	Cdk1	pAPC	Cdc20	\
0	0	0	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	0	
2	0	0	0	0	0	0	0	0	0	0	
3	0	0	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	0	0	
5	0	0	0	0	0	0	0	0	0	0	
6	0	0	0	0	0	0	0	0	0	0	
7	0	0	0	0	0	0	0	0	0	0	

	Cdh1	Replication	f4N_DNA	U_Kinetochores	Mad2	A_Kinetochores	Plk1_H	\
0	1		0	0	0		0	0
1	1		0	0	0		0	0
2	1		0	0	0		0	0
3	1		0	0	0		0	0
4	1		0	0	0		0	0
5	1		0	0	0		0	0
6	1		0	0	0		0	0
7	1		0	1	0		0	0

Ect2 Casp8 Casp2 MCL_1 BCLXL BCL2 BAD BIK BIM BID BAK BAX \

0	0	1	1	0	0	0	1	1	1	1	1	1
1	0	1	1	0	0	0	1	1	1	1	1	1
2	0	1	1	0	0	0	1	1	1	1	1	1
3	0	1	1	0	0	0	1	1	1	1	1	1
4	0	1	1	0	0	0	1	1	1	1	1	1
5	0	1	1	0	0	0	1	1	1	1	1	1
6	0	0	0	1	1	1	0	0	0	0	0	0
7	0	0	0	1	1	1	0	0	0	0	0	0

	Cyto_C	SMAC	IAPs	Casp9	Casp3	CAD	ATR	CHK1	Ca2	DAG	DR4_5	ERK	\
0	1	1	0	1	1	1	0	0	0	0	0	0	
1	1	1	0	1	1	1	0	0	0	0	1	0	
2	1	1	0	1	1	1	0	0	0	0	0	0	
3	1	1	0	1	1	1	0	0	0	0	0	0	
4	1	1	0	1	1	1	0	0	0	0	1	0	
5	1	1	0	1	1	1	0	0	0	0	1	0	
6	0	0	1	0	0	0	0	0	0	0	0	0	
7	0	0	1	0	0	0	0	0	0	0	0	0	

	GF_High	IP3	MEK	SOS	Trail
0	0	0	0	0	0
1	0	0	0	0	1
2	0	0	0	0	0
3	1	0	0	0	0
4	0	0	0	0	1
5	1	0	0	0	1
6	0	0	0	0	0
7	0	0	0	0	0

bioLQM returns eight *stable states* (also called *fixed points*), matching those reported by Sizek and colleagues in the Table S2 of their article.

To better visualise any of these stable states, the corresponding values can be projected on the model regulatory graph with a simple code line, using *GINsim*. Such a visualisation highlights the pathways/modules that are active in one particular stable state. An example of such stable state projection is provided in Figure 3.

```
[5]: # visualising the first fixed point from the table above.
ginsim.show(model_ginsim,fps[0])
```

```
[5]: <IPython.core.display.HTML object>
```

Here we have projected the first stable stated reported in the table above. Inactivated nodes (value 0) are shown in white, while activated nodes (value 1) are shown in blue.

1.6.4 3.4. Computation of synchronous attractors with *BNS*

In their publication, Sizek and collaborators report the existence of a cyclic attractor when a synchronous simulation method is used. Synchronous simulations simultaneously update all the

components at a given state, according to the logical rules, leading to a unique successor state. This is a strong assumption, as in reality, timing plays an important role in biology, and concurrent events barely occur simultaneously. Asynchronous simulations generally update only one component at each time step, either selecting randomly one component to update, or yet considering separately all single update calls, giving rise to complex, non deterministic dynamics. Of note, for a given Boolean model, the stable states are not affected by the choice of updating, as they bear no update calls. Cyclic attractors correspond to a circular sequence of events, such as the activation of cyclins, markers of the cell cycle. Cyclic attractors may depend on the choice of updating, and their identification is more complex in the asynchronous updating case.

BNS can be used to compute the synchronous attractors of the model (i.e., fixed points as well as simple cyclic trajectories in the synchronous case). Although *BNS* can search for attractors of any length, this search fails with such a complex model. To overcome this limitation, a range of cycle lengths is defined and attractors for the corresponding lengths are explored sequentially.

As Sizek et al. reported a synchronous cycle of length 21 in their Table S2, we make sure to include this value in the interval considered.

Of note, an attractor of length k is also an attractor of length $2k$, $3k$, etc. In particular, fixed points are detected as attractors of any length. Hence, in the following code cell, we consider a maximum length of 32 and filter the attractors amounting to iterations of smaller ones.

```
[6]: # Code to compute synchronous attractors of lengths smaller or equal to a
      ↪ user-defined maximum length
def bns_attractors(model, max_length):
    bns_attractors = []
    for l in range(1,max_length):
        attractors = bns.attractors(model, length=l)
        count = len(attractors)

        # remove attractors which are already known
        for pl, pcount in bns_attractors:
            if l % pl == 0:
                count -= pcount
                if count < 1: break

        if count > 0:
            bns_attractors.append( (l,count) )
            print(f'The model has {count} synchronous cyclic attractors of
            ↪ length {l}')

```

```
[7]: %time bns_attractors(model_biolqm, 32) # max-length is set to 32

```

```
The model has 8 synchronous cyclic attractors of length 1
The model has 4 synchronous cyclic attractors of length 10
The model has 1 synchronous cyclic attractors of length 21
CPU times: user 1.5 s, sys: 123 ms, total: 1.62 s
Wall time: 28.9 s

```

In addition to the eight fixed points already identified (corresponding to cyclic attractors of length

1), *BNS* returns four attractors of length 10, and one attractor of length 21, which correspond to the cyclic attractors reported by Sizek et al. in their supplementary Table S2.

Of note, attractors of lengths higher than 32 cannot be excluded by this analysis, as we use this number as a limit length to avoid long computations.

1.6.5 3.5. Analysis of the model constitutive modules

To generate their complex model, Sizek and colleagues initially developed smaller models for different functional modules (cf. Introduction). These modules correspond to processes such as DNA damage, cell cycle, MAPK pathway, or apoptosis, and encompass the main genes that are known to be involved in these processes. They are provided by the authors in their supplementary material, and we stored this information in a zip file available on github. The following code cell retrieves this information and counts the total number of components in each module, as well as the number of regulators of the module (aka inputs). It then computes the number of stable states, the number of terminal trap spaces (i.e. sub-hypercubes closed by the logical rules, comprising the stable states, but potentially also approximations to cyclic attractors), as well as the numbers of synchronous and asynchronous attractors for each of these modules.

```
[8]: # Download module files
if not os.path.isdir("Sizek_modules"):
    zf = "Sizek_modules.zip"
    if not os.path.exists(zf):
        print("Downloading", zf)
        from urllib.request import urlretrieve
        urlretrieve(f"https://gist.github.com/pauleve/
↪6cbd2358e24086c15cfd2ecf5b2e35e7/raw/
↪2750ef470c88a3d5044e3b4d76c2e4d31a84aa1b/{zf}",
                    filename=zf)
    print("Extracting", zf)
    from zipfile import ZipFile
    with ZipFile(zf, 'r') as z:
        z.extractall(".")
```

```
[9]: def extract_module(full, name, m):
    cpts = [ n.getNodeID() for n in m.getComponents() ]
    mred = biolqm.submodel(full, " ".join(cpts))
    cpts = [ n.getNodeID() for n in mred.getComponents() ]
    biolqm.save(mred, "Sizek_modules/%s.mnet" % name)
    return mred
```

```
[10]: %%time
summary = {}
for f in os.listdir("Sizek_modules"):
    if f.endswith(".txt"):
        name = f[:-4]
        if name.startswith("full"): continue
        mdl = biolqm.load("Sizek_modules/%s.txt" % name, "booleannet")
```

```

size = len( mdl.getComponents() )
fps = biolqm.fixpoints(mdl, autoconvert=False)
trp = biolqm.trapspace(mdl, autoconvert=False)
a_bns = bns.attractors(mdl)
a_bsm = boolsim.attractors(mdl)
mred = extract_module(model_biolqm, name, mdl)
summary[name] = {
    "size": size,
    "inputs": mred.getComponents().size() - size,
    "fps": fps.size(),
    "traps": trp.size(),
    "sync": len(a_bns),
    "async": len(a_bsm)
}
pd.DataFrame(summary)

```

CPU times: user 52.4 ms, sys: 18.6 ms, total: 71.1 ms

Wall time: 658 ms

```

[10]:
Restriction_Switch  Apoptotic_Switch  Phase_Switch  PI3K  \
size                7                16             15    8
inputs             14                13             16   11
fps                 2                  2              3    0
traps               2                  2              3    1
sync                2                  2              3    1
async               2                  2              3    1

Origin_Licensing_Switch
size                4
inputs             11
fps                 2
traps               2
sync                2
async               2

```

We can observe that the modules are highly interconnected, as they all have high numbers of inputs relative to their sizes, due to regulations exerted by nodes belonging to other modules.

Furthermore, all these modules but one display multistability but no cyclic behaviour. The PI3K module is the exception, as it has a unique, cyclic attractor, regardless of the use of synchronous or asynchronous updating. The cyclic behaviour of the PI3K module is indeed reported and discussed by Sizek et al.

This analysis shows that the attractors of the modules cannot explain the attractors of the model and that the combination of these modules gives rise to new behaviours. For instance, at this point, it becomes apparent that the global cyclic behaviour corresponding to the cell cycle (see section 3.4) arises from the interconnections of multiple modules.

1.6.6 3.6 Characterisation of synchronous cyclic properties with bioLQM

To interpret the meaning of the cyclic attractors, it is important to check which variables are fixed in this attractor and which ones switch values.

To visualise the 21 states composing the synchronous cyclic attractor, *bioLQM* can be used to generate a trace from one of the cycle states reported by Sizek et al., considering a number of steps somewhat greater than 21.

First, we define an initial state belonging to the cyclic attractor, according to Sizek et al. with the following code cell:

```
[11]: init_active = {'AKT_B',  
    'BCL2',  
    'BCLXL',  
    'Cdc20',  
    'Cdc25A',  
    'Cdc25B',  
    'Cdc25C',  
    'Cdh1',  
    'CyclinA_mRNA',  
    'CyclinD1',  
    'E2F1',  
    'ERK',  
    'Emi1',  
    'GF',  
    'GF_High',  
    'Grb2',  
    'IAPs',  
    'MCL_1',  
    'MEK',  
    'Myc',  
    'ORC',  
    'PDK1',  
    'PI3K',  
    'PIP3',  
    'Plk1_H',  
    'RAF',  
    'RTK',  
    'Ras',  
    'S6K',  
    'SOS',  
    'TSC2',  
    'UbcH10',  
    'eIF4E',  
    'f4N_DNA',  
    'mTORC1',  
    'mTORC2',  
    'pAPC'}
```

```
# dict-like representation, associating a node to either 0 or 1
init = {n: 1 if n in init_active else 0 for n in map(str, model_biolqm.
    ↳getComponents())}
# str-like representation, for bioLQM
init_str = "".join(map(str, (init[n] for n in map(str, model_biolqm.
    ↳getComponents()))))
init_str
```

```
[11]: '11111111110000000001001110000111010000110100101010111010001000111000000001000000
00110110'
```

Next, we compute a synchronous updating trace, starting with this initial state and considering a maximum of 50 steps, with the following code cell.

```
[12]: %%time
# The trace method of bioLQM typically takes the following arguments:
# -i - set of initial conditions specified for each node
# -m - maximum number of steps after which the simulation will stop
# -u - updating type (synchronous by default, can also be sequential where a
    ↳certain updating order is prioritised)

trace = biolqm.trace(model_biolqm, f"-i {init_str} -m 50")
trace_df = pd.DataFrame( [s for s in trace] )
trace_df
```

CPU times: user 121 ms, sys: 72 ms, total: 193 ms

Wall time: 301 ms

```
[12]:
```

	GF	RTK	Grb2	Ras	RAF	mTORC2	PI3K	PIP3	PDK1	AKT_B	p110_H	PI3K_H	\
0	1	1	1	1	1	1	1	1	1	1	0	0	
1	1	1	1	1	1	1	1	1	1	1	0	0	
2	1	1	1	1	1	1	1	1	1	1	1	0	
3	1	1	1	1	1	1	1	1	1	1	1	1	
4	1	1	1	1	1	1	1	1	1	1	1	1	
5	1	1	1	1	1	1	1	1	1	1	1	1	
6	1	1	1	1	1	1	1	1	1	1	1	1	
7	1	1	1	1	1	1	1	1	1	1	1	1	
8	1	1	1	1	1	1	1	1	1	1	0	1	
9	1	1	1	1	1	1	1	1	1	1	0	0	
10	1	1	1	1	1	1	1	1	1	1	0	0	
11	1	1	1	1	1	1	1	1	1	1	0	0	
12	1	1	1	1	1	1	1	1	1	1	1	0	
13	1	1	1	1	1	1	1	1	1	1	1	1	
14	1	1	1	1	1	1	1	1	1	1	1	1	
15	1	1	1	1	1	1	1	1	1	1	1	1	
16	1	1	1	1	1	1	1	1	1	1	1	1	
17	1	1	1	1	1	1	1	1	1	1	1	1	

18	1	1	1	1	1	1	1	1	1	1	0	1
19	1	1	1	1	1	1	1	1	1	1	0	0
20	1	1	1	1	1	1	1	1	1	1	0	0
21	1	1	1	1	1	1	1	1	1	1	0	0

	AKT_H	FoxO3	PLCgamma	NeddL4	FoxO1	p21_mRNA	TSC2	PRAS40	Rheb	\
0	0	0	0	0	0	0	1	0	0	
1	0	1	0	0	1	0	1	0	0	
2	0	1	0	0	1	1	1	0	0	
3	0	1	0	0	1	1	1	0	0	
4	1	1	1	0	1	1	1	0	0	
5	1	0	1	0	0	1	0	0	0	
6	1	0	1	0	0	0	0	0	1	
7	1	0	1	1	0	0	0	0	1	
8	1	0	1	1	0	0	0	0	1	
9	0	0	0	1	0	0	0	0	1	
10	0	1	0	1	1	0	1	0	1	
11	0	1	0	0	1	1	1	0	0	
12	0	1	0	0	1	1	1	0	0	
13	0	1	0	0	1	1	1	1	0	
14	1	1	1	0	0	1	1	1	0	
15	1	0	1	0	0	0	0	0	0	
16	1	0	1	0	0	0	0	0	1	
17	1	0	1	1	0	0	0	0	1	
18	1	0	1	1	0	0	0	0	1	
19	0	0	0	1	0	0	0	0	1	
20	0	0	0	1	0	0	1	0	1	
21	0	0	0	0	0	0	1	0	0	

	mTORC1	S6K	eIF4E	GSK3	p21	pRB	p27Kip1	Myc	CyclinD1	E2F1	CyclinE	\
0	1	1	1	0	0	0	0	1	1	1	0	
1	1	1	1	0	0	0	0	1	1	1	0	
2	1	1	1	0	0	0	0	1	1	1	0	
3	1	1	1	0	1	0	0	1	1	1	0	
4	1	1	1	0	1	0	0	1	1	1	1	
5	1	1	1	0	0	0	0	1	1	1	1	
6	1	1	1	0	0	0	0	1	1	1	1	
7	1	1	1	0	0	0	0	1	1	1	1	
8	1	1	1	0	0	0	0	1	1	0	1	
9	1	1	1	0	0	0	0	1	1	0	0	
10	1	1	1	0	0	0	0	1	0	0	0	
11	1	1	1	0	0	0	1	1	0	0	0	
12	0	1	1	0	1	0	1	1	1	0	0	
13	0	0	0	0	1	0	0	1	0	0	0	
14	0	0	0	1	1	0	1	1	0	0	0	
15	0	0	0	0	1	0	0	0	0	0	0	
16	0	0	0	0	0	0	0	1	0	0	0	

17	1	0	0	0	0	0	0	1	1	0	0
18	1	1	1	0	0	0	0	1	1	1	0
19	1	1	1	0	0	0	0	1	1	1	0
20	1	1	1	0	0	0	0	1	1	1	0
21	1	1	1	0	0	0	0	1	1	1	0

	ORC	Cdc6	Cdt1	Pre_RC	geminin	CyclinA_mRNA	Emi1	FoxM1	Cdc25A	\
0	1	0	0	0	0	1	1	0	1	
1	1	1	0	0	0	1	1	0	0	
2	1	1	1	0	0	1	1	0	0	
3	1	1	1	1	0	1	1	0	0	
4	1	1	1	1	0	1	1	0	0	
5	1	1	1	1	0	1	1	1	0	
6	1	1	1	1	0	1	1	1	1	
7	1	1	1	1	0	1	1	1	1	
8	1	1	0	1	0	1	1	1	1	
9	0	0	0	0	0	1	1	1	1	
10	0	0	0	0	0	1	1	1	1	
11	0	0	0	0	0	1	1	1	1	
12	0	0	0	0	0	1	1	1	1	
13	0	0	0	0	0	1	1	1	1	
14	0	0	0	0	0	1	1	1	1	
15	0	0	0	0	0	1	1	1	1	
16	0	0	0	0	0	1	1	1	1	
17	0	0	0	0	0	1	0	1	1	
18	0	0	0	0	0	1	0	1	1	
19	1	0	0	0	1	1	0	1	1	
20	1	0	0	0	0	1	0	1	1	
21	1	0	0	0	0	1	1	0	1	

	CyclinA	Wee1	UbcH10	CyclinB	Cdc25B	Plk1	Cdc25C	Cdk1	pAPC	Cdc20	\
0	0	0	1	0	1	0	1	0	1	1	
1	0	0	1	0	0	0	0	0	1	0	
2	0	0	0	0	0	0	0	0	0	0	
3	0	0	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	0	0	
5	0	0	0	0	0	0	0	0	0	0	
6	0	0	0	0	0	0	0	0	0	0	
7	1	0	0	0	0	0	0	0	0	0	
8	1	1	0	0	0	0	0	0	0	0	
9	1	1	1	1	1	0	0	0	0	0	
10	1	1	1	1	1	0	0	0	0	0	
11	1	1	1	1	1	0	0	0	0	0	
12	1	0	1	1	1	0	0	0	0	0	
13	1	0	1	1	1	1	0	0	0	0	
14	1	0	1	1	1	1	1	0	0	0	
15	1	0	1	1	1	1	1	1	0	0	

16	1	0	1	1	1	1	1	1	1	0
17	0	0	1	1	1	1	1	1	1	0
18	0	0	1	1	1	1	1	1	1	0
19	0	0	1	1	1	1	1	1	1	1
20	0	0	1	0	1	1	1	1	1	1
21	0	0	1	0	1	0	1	0	1	1

	Cdh1	Replication	f4N_DNA	U_Kinetochores	Mad2	A_Kinetochores	Plk1_H	\
0	1	0	1	0	0	0	1	
1	1	0	1	0	0	0	0	
2	1	0	0	0	0	0	0	
3	1	0	0	0	0	0	0	
4	1	0	0	0	0	0	0	
5	1	0	0	0	0	0	0	
6	1	0	0	0	0	0	0	
7	1	1	0	0	0	0	0	
8	0	1	1	0	0	0	0	
9	0	0	1	0	0	0	0	
10	0	0	1	0	0	0	0	
11	0	0	1	0	0	0	0	
12	0	0	1	0	0	0	0	
13	0	0	1	0	0	0	0	
14	0	0	1	0	0	0	1	
15	0	0	1	0	0	0	1	
16	0	0	1	1	0	0	1	
17	0	0	1	1	1	1	1	
18	0	0	1	0	0	1	1	
19	0	0	1	0	0	1	1	
20	0	0	1	0	0	0	1	
21	1	0	1	0	0	0	1	

	Ect2	Casp8	Casp2	MCL_1	BCLXL	BCL2	BAD	BIK	BIM	BID	BAK	BAX	\
0	0	0	0	1	1	1	0	0	0	0	0	0	
1	1	0	0	1	1	1	0	0	0	0	0	0	
2	0	0	0	1	1	1	0	0	0	0	0	0	
3	0	0	0	1	1	1	0	0	0	0	0	0	
4	0	0	0	1	1	1	0	0	0	0	0	0	
5	0	0	0	1	1	1	0	0	0	0	0	0	
6	0	0	0	1	1	1	0	0	0	0	0	0	
7	0	0	0	1	1	1	0	0	0	0	0	0	
8	0	0	0	1	1	1	0	0	0	0	0	0	
9	0	0	0	1	1	1	0	0	0	0	0	0	
10	0	0	0	1	1	1	0	0	0	0	0	0	
11	0	0	0	1	1	1	0	0	0	0	0	0	
12	0	0	0	1	1	1	0	0	0	0	0	0	
13	0	0	0	1	1	1	0	0	0	0	0	0	
14	0	0	0	1	1	1	0	0	0	0	0	0	

15	0	0	0	1	1	1	0	0	0	0	0	0
16	0	0	0	1	1	1	0	0	0	0	0	0
17	0	0	0	0	1	1	0	0	0	0	0	0
18	0	0	0	0	0	1	0	0	0	0	0	0
19	0	0	0	1	1	1	0	0	0	0	0	0
20	0	0	0	1	1	1	0	0	0	0	0	0
21	0	0	0	1	1	1	0	0	0	0	0	0

	Cyto_C	SMAC	IAPs	Casp9	Casp3	CAD	ATR	CHK1	Ca2	DAG	DR4_5	ERK	\
0	0	0	1	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	0	0	0	1
2	0	0	1	0	0	0	0	0	0	0	0	0	1
3	0	0	1	0	0	0	0	0	0	0	0	0	1
4	0	0	1	0	0	0	0	0	0	0	0	0	1
5	0	0	1	0	0	0	0	0	0	1	0	0	1
6	0	0	1	0	0	0	0	0	1	1	0	0	1
7	0	0	1	0	0	0	0	0	1	1	0	0	1
8	0	0	1	0	0	0	1	0	1	1	0	0	1
9	0	0	1	0	0	0	1	1	1	1	0	0	1
10	0	0	1	0	0	0	0	1	1	0	0	0	1
11	0	0	1	0	0	0	0	0	0	0	0	0	1
12	0	0	1	0	0	0	0	0	0	0	0	0	1
13	0	0	1	0	0	0	0	0	0	0	0	0	1
14	0	0	1	0	0	0	0	0	0	0	0	0	1
15	0	0	1	0	0	0	0	0	0	1	0	0	1
16	0	0	1	0	0	0	0	0	1	1	0	0	1
17	0	0	1	0	0	0	0	0	1	1	0	0	1
18	0	0	1	0	0	0	0	0	1	1	0	0	1
19	0	0	1	0	0	0	0	0	1	1	0	0	1
20	0	0	1	0	0	0	0	0	1	0	0	0	1
21	0	0	1	0	0	0	0	0	0	0	0	0	1

	GF_High	IP3	MEK	SOS	Trail
0	1	0	1	1	0
1	1	0	1	1	0
2	1	0	1	1	0
3	1	0	1	1	0
4	1	0	1	1	0
5	1	1	1	1	0
6	1	1	1	1	0
7	1	1	1	1	0
8	1	1	1	1	0
9	1	1	1	1	0
10	1	0	1	1	0
11	1	0	1	1	0
12	1	0	1	1	0
13	1	0	1	1	0

14	1	0	1	1	0
15	1	1	1	1	0
16	1	1	1	1	0
17	1	1	1	1	0
18	1	1	1	1	0
19	1	1	1	1	0
20	1	0	1	1	0
21	1	0	1	1	0

The execution of this cell generates a table encompassing 22 rows (cf. notebook), each corresponding to one state belonging to a cyclic sequence. We can easily check that the first and last states listed in the table above are indeed identical with a few lines of python code.

```
[13]: first_row = trace_df.iloc[0]
      last_row = trace_df.iloc[-1]
      if first_row.equals(last_row):
          print("The first state of the trace is equal to the last one")
```

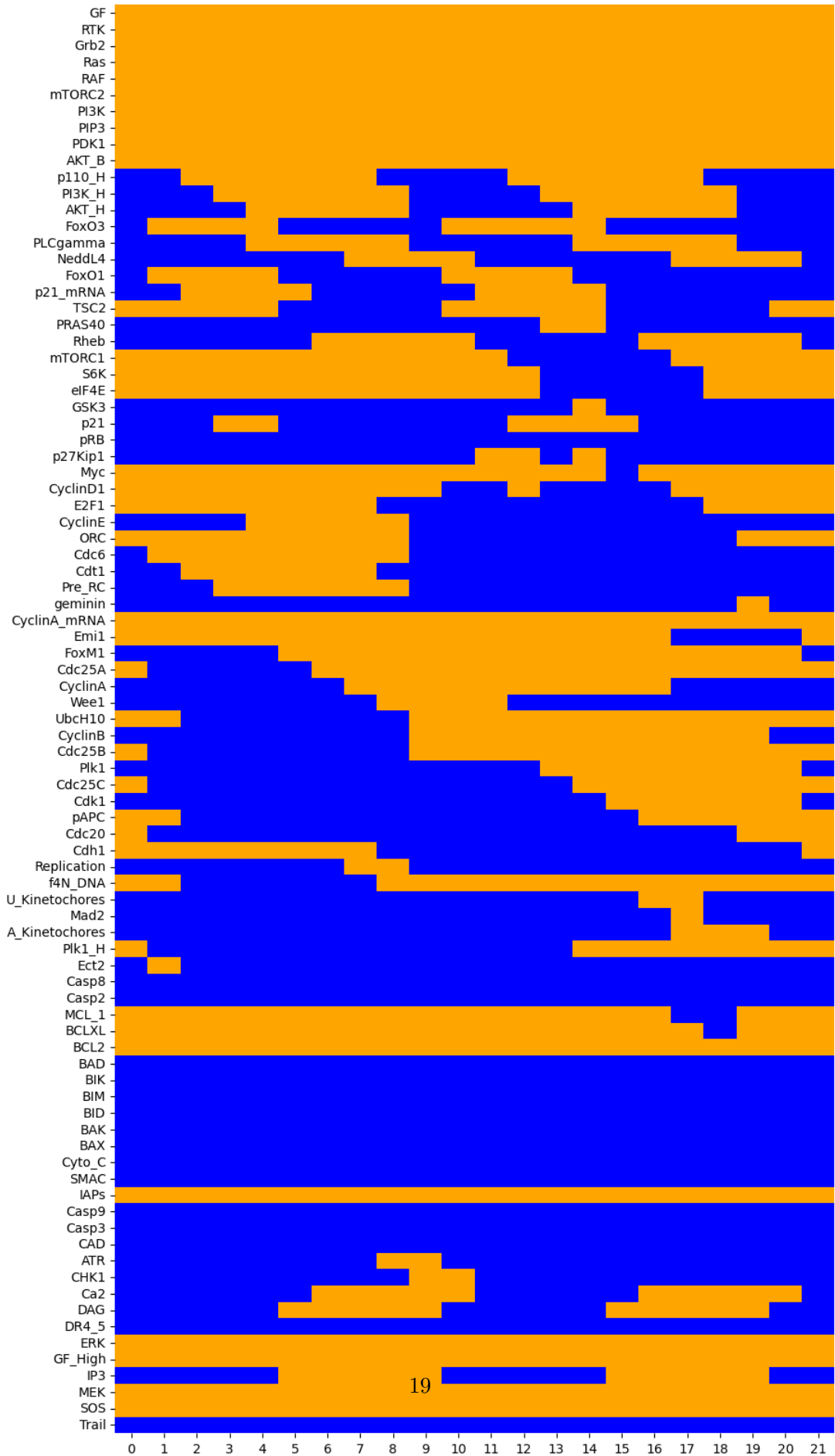
The first state of the trace is equal to the last one

The execution of this cell confirms that we have identified the cyclic attractor of length 21 mentioned above.

To ease the comparison of this sequence of states with the periodic pattern reported by Sizek et al. in their Figure 4, we can plot it with similar graphical conventions, using the following python code.

```
[14]: #Function used for plotting the dynamics of molecular species
      def plotting(df):
          custom_cmap = sns.color_palette(['blue', 'orange'])
          fig = plt.figure(figsize=(10, 20)) # Adjust the figure size as needed
          sns.heatmap(df.T, cmap=custom_cmap, annot=False, cbar=False)
          plt.show()

      plotting(trace_df)
```

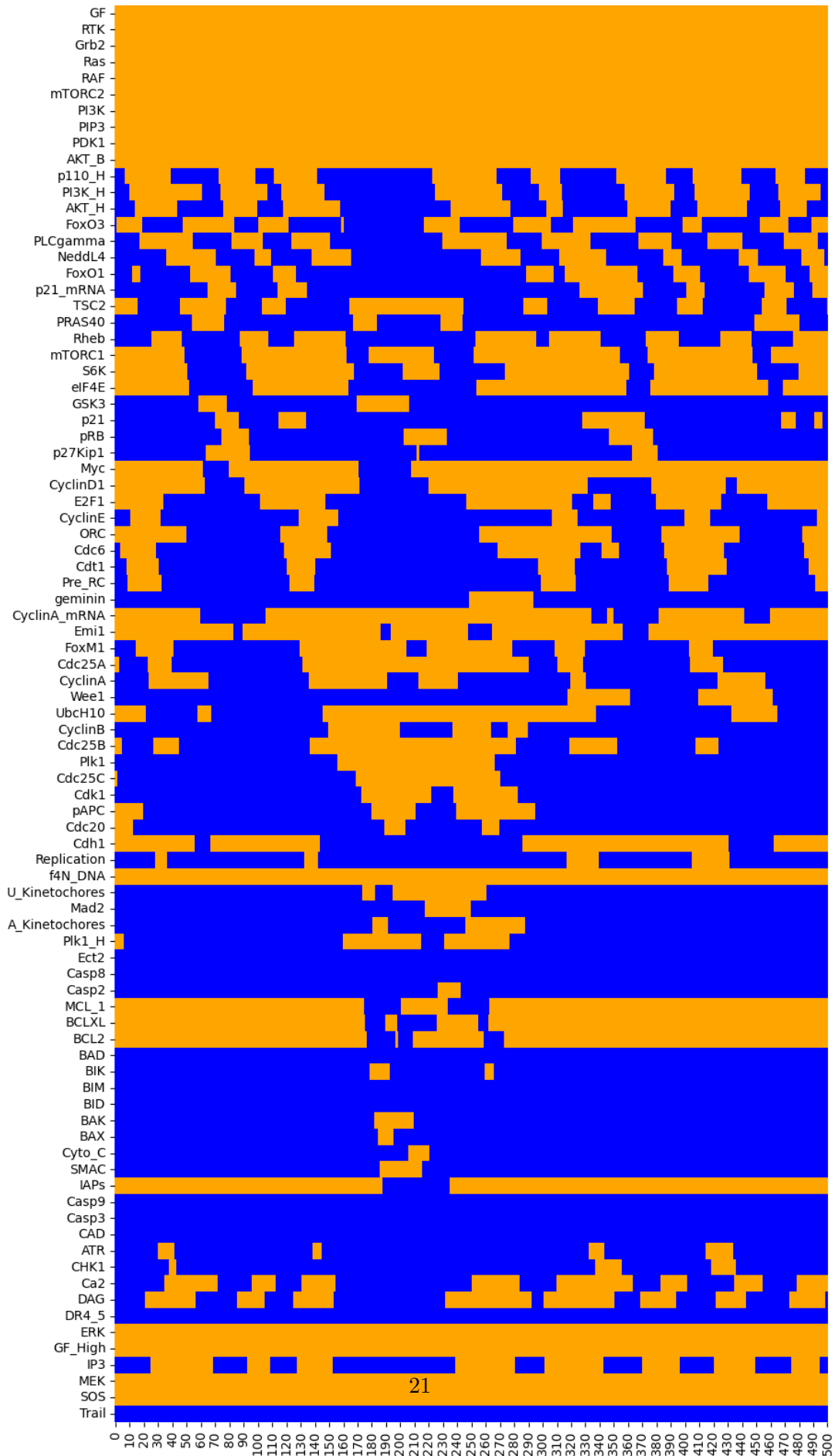


This heat map displays a sequence of 22 states (columns) corresponding to the synchronous cycle of length 21, with the initial state (first column, index 0) matching the last one (column index 21). Orange cells denote active components (rows) at the corresponding cycle state (column), whereas blue cells denote suppressed components. Note that some nodes are never activated along the cycle, while others are always activated. In particular, the apoptotic pathway (BIM, BAX, BAK, CytC, Casp3, Casp9, etc.) is inactive in this sequence, while the cell cycle is activated by the MAPK pathway (RTK, Ras, Raf, PI3K, etc.).

Hence, using different tools, we could also identify the synchronous attractor reported by Sizek et al., which recapitulates the typical sequence of (in)activation of cell components, including cyclins. Further analysis of the timing of cyclins (in)activations is provided in section 3.7 below.

Of note, Sizek et al. reported that this cyclic behaviour is sensitive to the updating strategy used. To further characterise this sensitivity, *bioLQM* can be used to compute traces starting from the same initial state but using random asynchronous updating (i.e., selecting one node among all nodes called to be updated at each state, with identical probabilities assigned to all single node transitions). This can be achieved using the following code cell.

```
[15]: %%time
# Random asynchronous trace considering a maximum of 500 steps
random = biolqm.random(model_biolqm, f"-i {init_str} -m 500")
random_df = pd.DataFrame( [s for s in random] )
plotting(random_df)
```



CPU times: user 3.34 s, sys: 1.32 s, total: 4.66 s
Wall time: 6.53 s

Note that a longer maximal number of steps (500) is considered here because we do not know if the trace will end up in an asynchronous cycle, or how long such a cycle might be.

In this heatmap, we can observe a transient oscillating pattern, but this pattern gets progressively degraded along the simulation. This is due to the random choice of transitions whenever several components are called to switch their values at a given state, according to their logical rules. Of note, if we repeat the same random asynchronous simulation, we typically end up with a different transient oscillating pattern.

Interestingly, when bigger enough maximal step numbers are considered, simulations ultimately lead to a stable state corresponding to an apoptotic fate. To visualise this, in the notebook, we provide the code for a simulation using a maximum number of steps set to 2000, which needs to be uncommented to be run.

```
[16]: # %%time
# Random asynchronous trace considering a maximum of 2000 steps
# This high number is chosen in order to visualise the derivation toward a
# ↪ stable state
# random = biolqm.random(model_biolqm, f"-i {init_str} -m 2000")
# random_df = pd.DataFrame( [s for s in random] )
# plotting(random_df)
```

In their manuscript, the authors defined a biased transition order to enforce a more robust cyclic behaviour.

Hereafter, we propose an alternative approach to deepen our understanding of the asynchronous behaviour of the model, taking advantage of the software MaBoSS [17] to integrate over a large number of potentially different traces.

1.6.7 3.7. Using MaBoSS software to perform stochastic simulations of the wild type model

It is difficult to build the asynchronous state transition graph and analyse the transient dynamics of such complex models. To cope with this issue, we use MaBoSS, a tool performing stochastic simulations over Boolean networks based on the continuous-time Markov chain formalism, relying on the Gillespie algorithm [17]. The software enables the generation of time plots showing the evolution of mean component values (or patterns thereof) and pie charts showing the probabilities of the different model states reached at the end of the simulation.

The biolqm model can be converted into the format required by MaBoSS using *bioLQM*.

```
[17]: # Conversion of bioLQM model into the MaBoSS format
model_maboss = biolqm.to_maboss(model_biolqm)
```

After loading the model, simulation parameters can be modified.

```
[18]: # Delineation of MaBoSS simulation parameters
model_maboss.update_parameters(sample_count=5000,max_time=200, time_tick=0.5)
```

This command line defines the following simulation parameters: - Number of simulations: 5000. - Maximum length of the simulation (in arbitrary unit): 200. - Interval between two time points: 0.5.

Note that MaBoSS uses continuous-time Boolean modeling and requires the specification of a maximum simulation length and an interval between time points. All the stochastic simulations reported in the notebook use these same simulation parameters.

Next, we define a wild-type version of the model, called *WT*, and we select the *external variables*, i.e. the variables for which the values will be reported during the simulation. This is crucial to avoid combinatorial explosion.

```
[19]: WT = model_maboss.copy()
      ## Select outputs for visualization of the results
      WT.network.set_output(('CyclinB', 'Casp3', 'PI3K_H', 'E2F1', 'Replication', 'p21',
      ↪ 'RTK'))
```

Note that when exporting a model to MaBoSS with bioLQM or GINsim, all the initial values are set to 0 by default.

Hereafter, we assign to the model the initial conditions defined by Sizek et al. in their Figure 5.

```
[20]: # Define a state corresponding to Sizek's initial condition
active_init = [ k for (k,v) in init.items() if v==1 ]
maboss.set_nodes_istate(WT, active_init, [0,1])
```

MaBoSS simulation results can be graphically displayed as *time plots* showing the probabilities of active nodes or model states (vectors of active nodes) over time, or as *pie charts* showing the probabilities of the final states (i.e., the states reached at the end of the simulations). In brief, the trajectories plotted on these graphs represent mean trajectories of 5000 single trajectories focusing on a subset of seven selected nodes. It is possible to visualise the activity of each of these nodes over time, or the evolution of model states defined as configurations of active nodes, which can also be displayed as a pie chart for the last time point considered.

```
[21]: %%time
      # Run a MaBoSS simulation
      run_WT = WT.run()
```

CPU times: user 1.94 ms, sys: 1.2 ms, total: 3.14 ms

Wall time: 38 s

```
[22]: # Plot the trajectories of the WT simulation for selected nodes
      run_WT.get_nodes_probtraj().plot(legend=True)
      plt.title('WT node trajectories')
      ## Note that the model state <nil> means that none of the external variables
      ↪ are ON.

      # Plot the trajectories of the WT simulation with model states
```

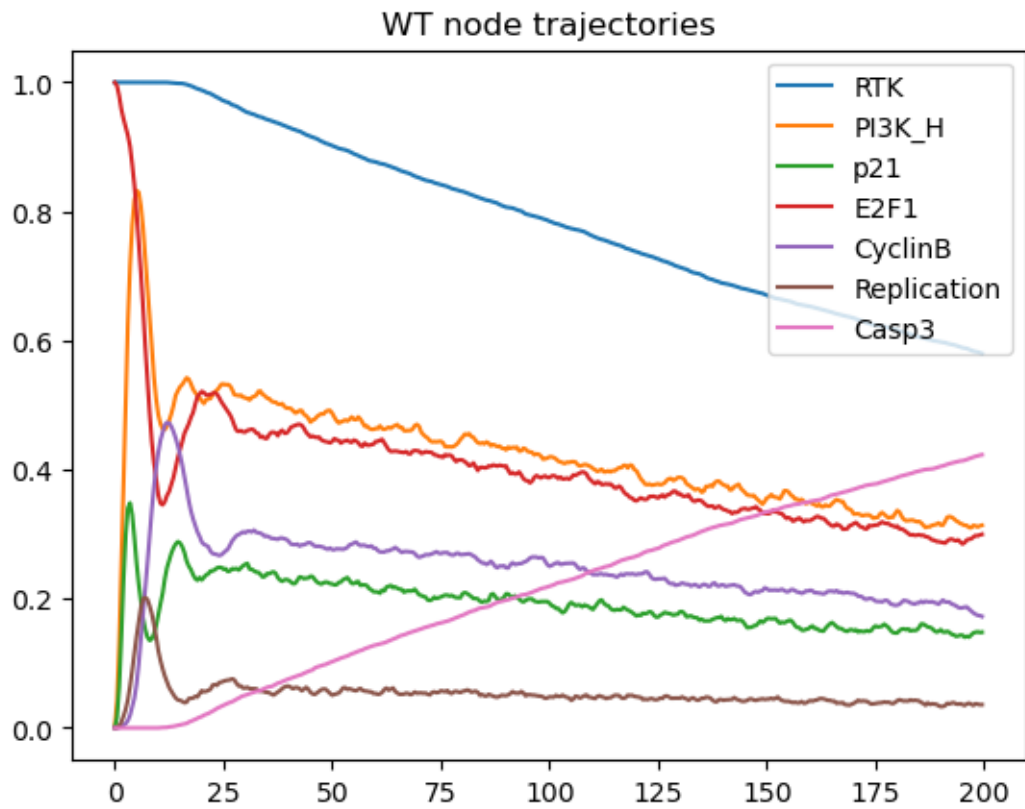
```

run_WT.plot_trajectory(legend=False)
plt.title('WT model state trajectories')

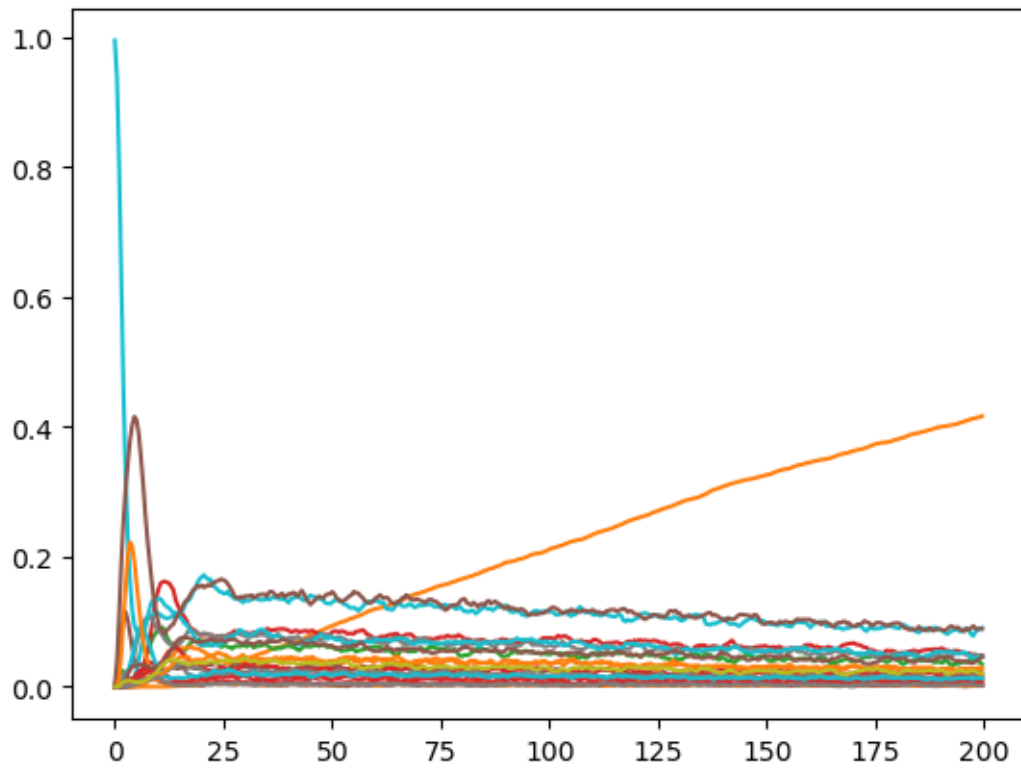
# Plot pie chart
run_WT.plot_piechart()
plt.title('Repartition of final WT model states')

```

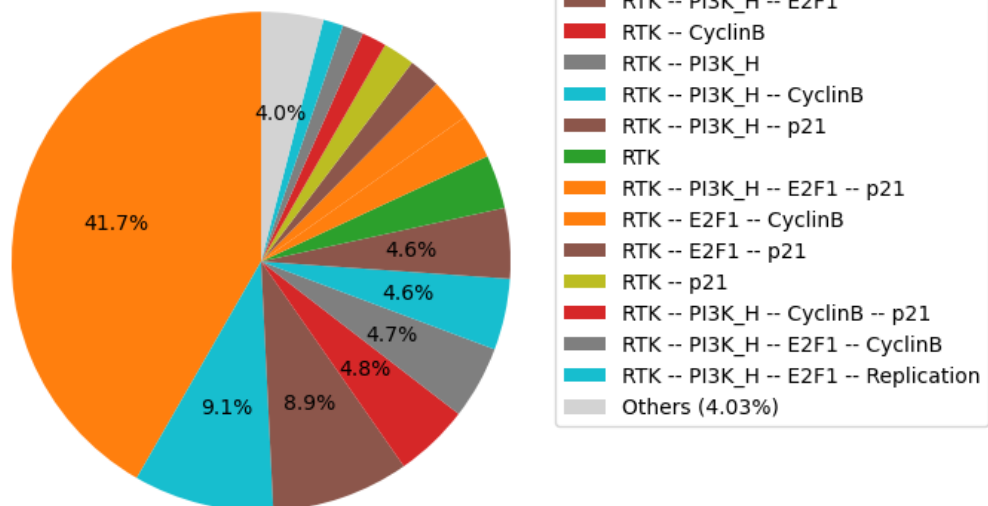
[22]: Text(0.5, 1.0, 'Repartition of final WT model states')



WT model state trajectories



Repartition of final WT model states

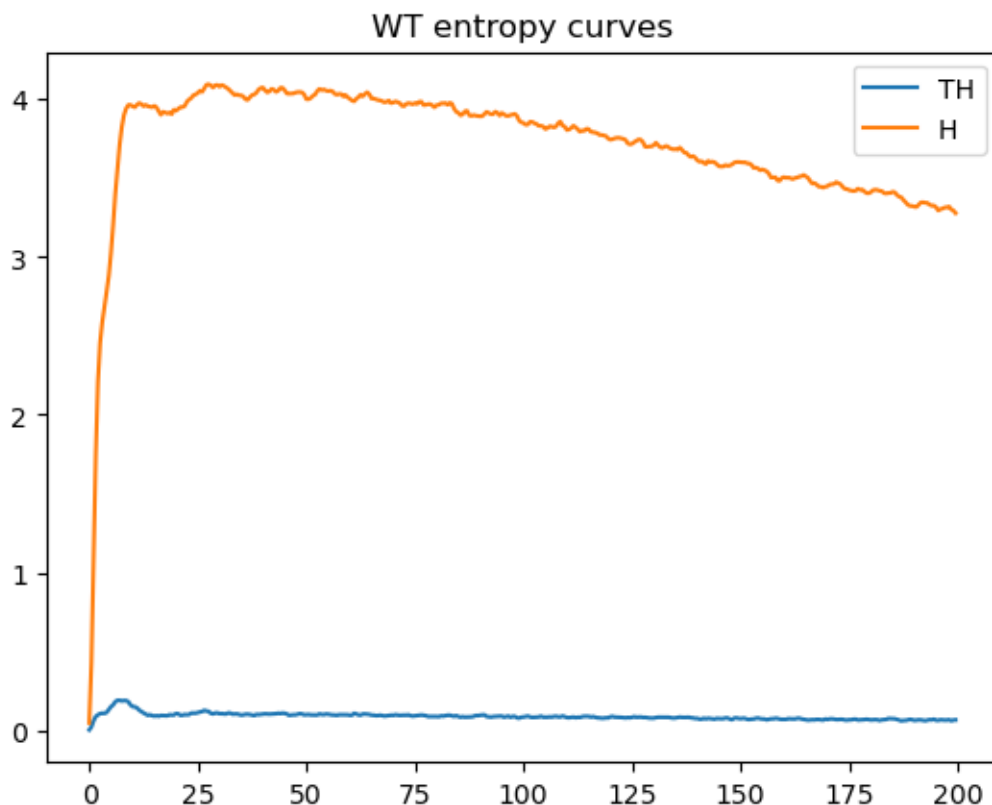


The simulations reveal a transient cyclic-like behaviour, which is eventually lost due to the spontaneous Casp3 activation, thereby revealing the progressive dominance of the apoptotic stable state.

It is also possible to plot the **state entropy** (H) and the **transition entropy** (TH) over time, which can be interpreted as signatures for the stable states or cyclic attractors. These curves provide a means to check the nature of the attractors of the model. Entropy measures the “disorder” of the system studied. A very low transition entropy at the end of a simulation thus suggests that the system has reached an equilibrium, while a positive state entropy may suggest the existence of a cyclic attractor or the occurrence of multiple stable states (for more details, see [18]).

```
[23]: # Plot the state and transition entropy curves
run_WT.plot_entropy_trajectory()
plt.title('WT entropy curves')
```

```
[23]: Text(0.5, 1.0, 'WT entropy curves')
```



The state entropy initially rises abruptly, and later slowly decreases. In parallel, the transition entropy curve displays a little peak before remaining close to zero, with some noise.

Together, these two curves are suggestive of a transient periodic behaviour, which ultimately collapses as more and more cells trigger apoptosis.

Next, we can use *MaBoSS* to study the impact of changes in the initial conditions on the dynamic

behaviour of the model. In the preceding simulation, all death signals were initially OFF. Hereafter, we assess the effect of activating TRAIL, a ligand binding to death receptors of the tumour cells and initiating the death pathways, while keeping the rest of the other initial conditions unchanged. Note that TRAIL is an input of the death module, and thus not regulated. Such inputs are set either to 0 or 1 in the initial conditions, thereby conveying the status of the environment.

The following code cells defines a copy of the WT model, triggers TRAIL level to 1 at the initial state, performs the simulation, and finally displays the results:

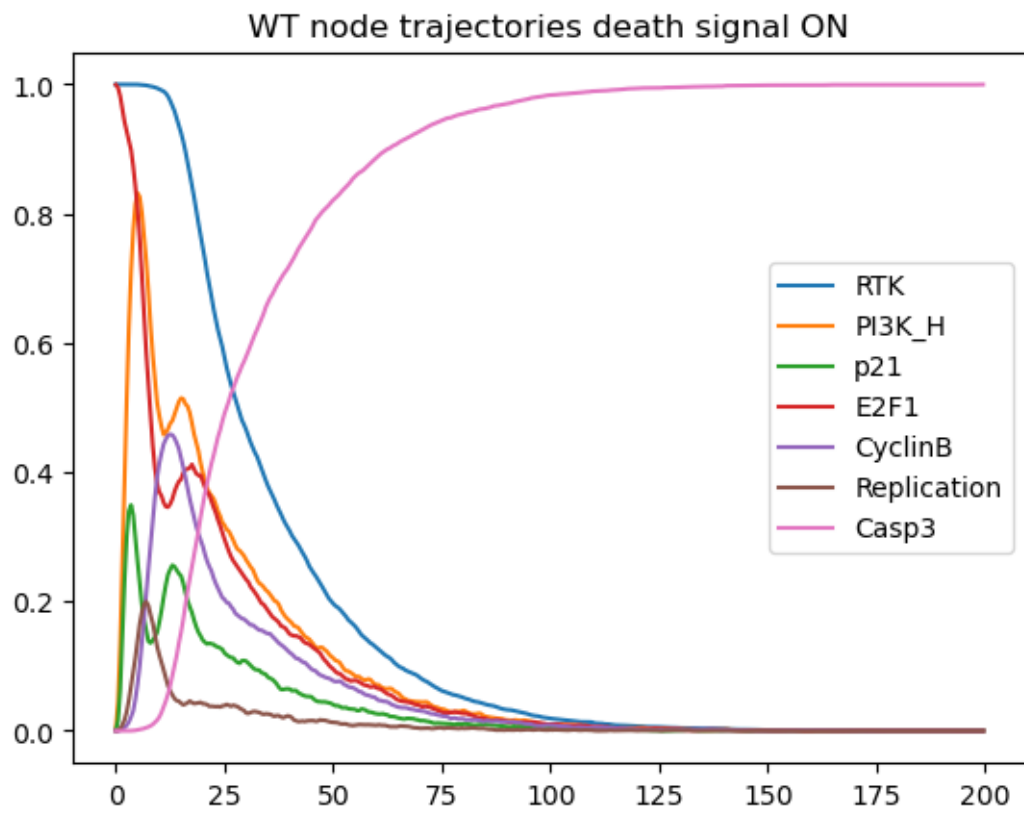
```
[24]: # Copy the WT model and change the value of Trail at the initial state
      WT_death = WT.copy()
      maboss.set_nodes_istate(WT_death, ["Trail" ] + active_init, [0,1])

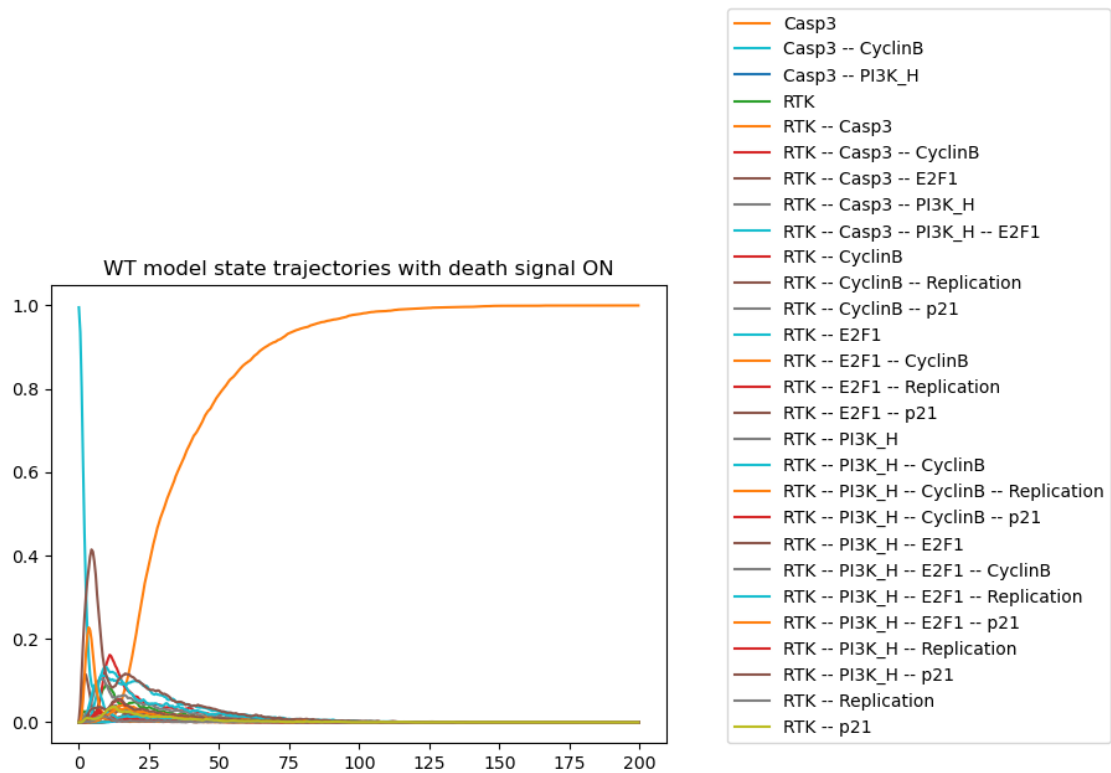
[25]: %%time
      # We repeat the MaBoSS simulations using the modified initial conditions.
      # Run a MaBoSS simulation
      WT_death.update_parameters(sample_count=5000,max_time=200, time_tick=0.5)
      run_WT_death = WT_death.run()

CPU times: user 2.07 ms, sys: 10 s, total: 2.08 ms
Wall time: 10.8 s

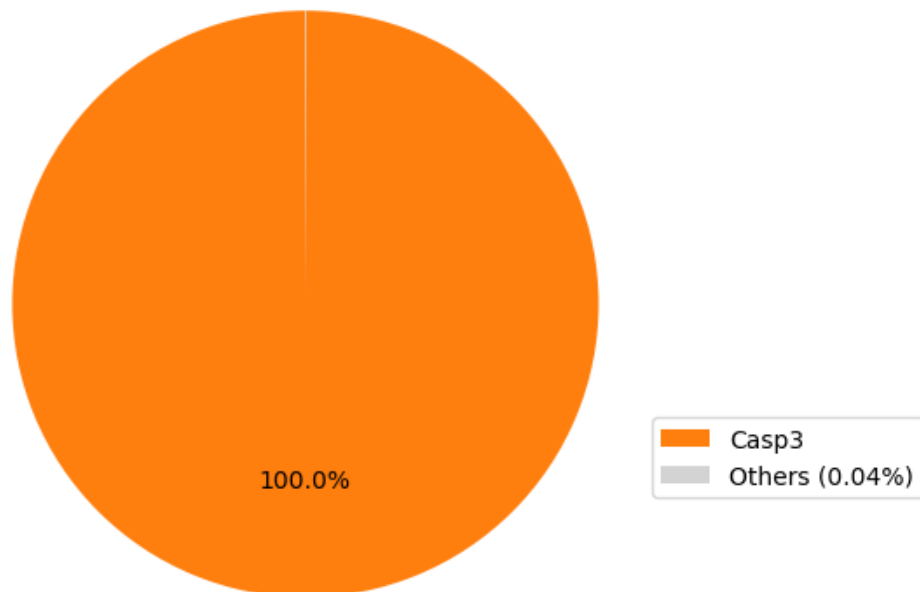
[26]: # Plot the results for the WT model with dead signal ON
      # Plot the WT node trajectories with dead signal ON
      #run_WT_death.plot_trajectory(legend=False)
      run_WT_death.get_nodes_probtraj().plot(legend=True)
      plt.title('WT node trajectories death signal ON')
      # Plot the WT model state trajectories with dead signal ON
      run_WT_death.plot_trajectory(legend=True)
      plt.title('WT model state trajectories with death signal ON')
      # Plot WT pie chart with dead signal ON
      run_WT_death.plot_piechart(legend=True)
      plt.title('Repartition of final states for the WT model with death signal ON')
      # Plot the WT state and transition entropy curves with dead signal ON
      run_WT_death.plot_entropy_trajectory()
      plt.title('Entropy curves for WT model with dead signal ON')
```

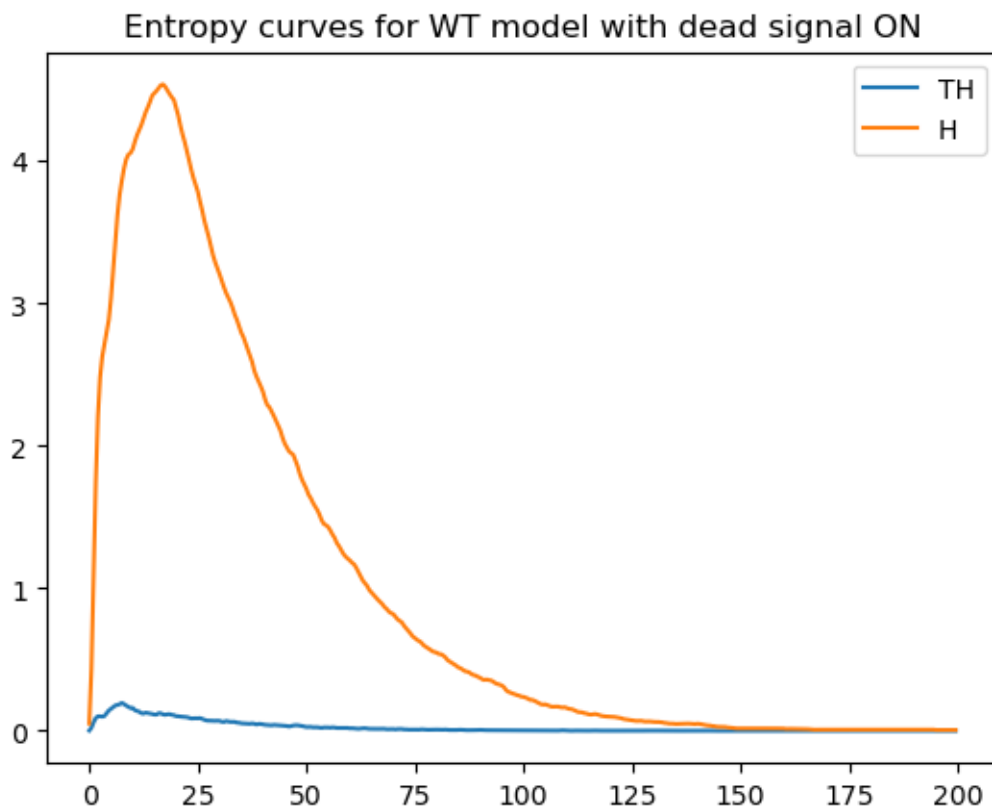
```
[26]: Text(0.5, 1.0, 'Entropy curves for WT model with dead signal ON')
```





Repartition of final states for the WT model with death signal ON





The activation of Trail at the initial state enables a faster rise of Casp3 and thus of the apoptotic fate. In parallel, the state entropy drops much faster, while the transition entropy stabilises around zero, suggesting a stable state situation.

1.6.8 3.8. Mutant simulations with MaBoSS

In their article, Sizek and colleagues report a sophisticated analysis of the role and dynamical properties of the PI3K/AKT1 pathway, and recapitulate deleterious effects of its alterations. They further analyse the impact of the blocking of the Polo-like kinase 1 (Plk1, a mitotic driver and chemotherapy target) on cell cycle progression.

Hereafter, we focus on the analysis of the impact of the perturbation of other model components, using the *MaBoSS* software.

Such perturbations can be understood as mutations that patients bear, or as targets of drugs that would suppress proliferation or increase apoptosis.

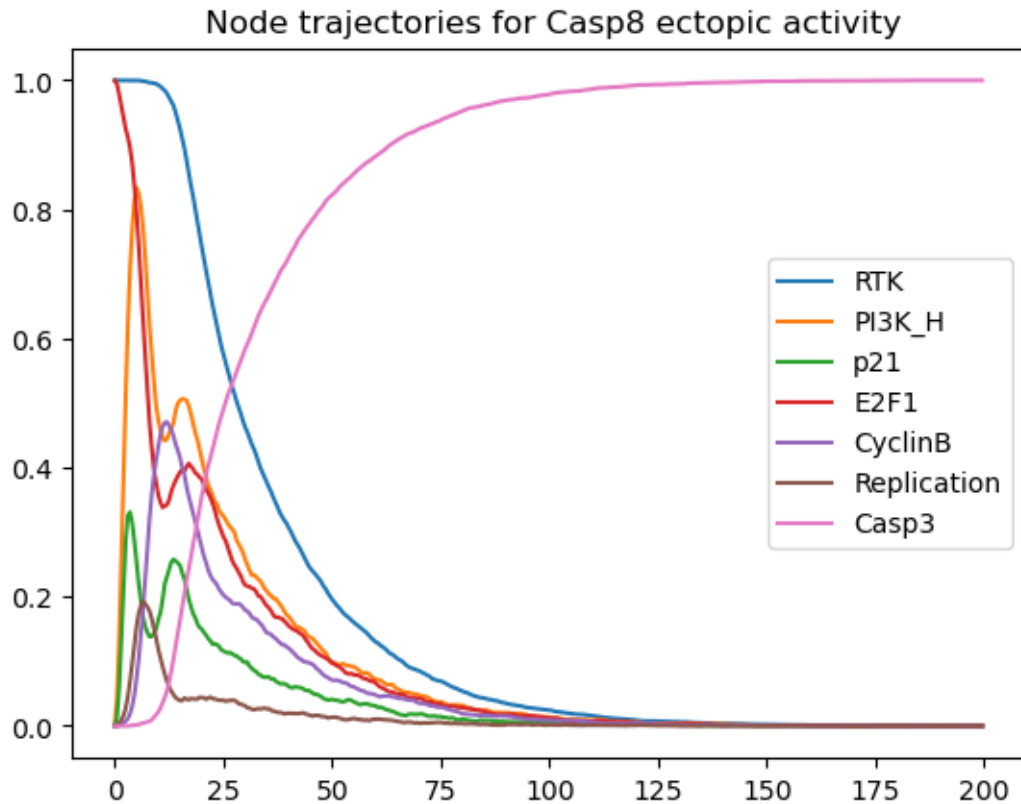
3.8.1. Stochastic simulation of ectopic Casp8 activity Casp8 is a caspase initiating the apoptotic cascade. By changing its activity (ectopic activity or knock-down), we can explore its role in triggering cell death. Using the following code cell, we first simulate the impact of a constant activation of Casp8 and display the resulting time plot and pie chart.

```
[27]: %%time
# MaBoSS simulation for Casp8 ectopic activity
mut_Casp8_ON = WT.copy()
mut_Casp8_ON.mutate("Casp8", "ON")
mutres_Casp8_ON = mut_Casp8_ON.run()
```

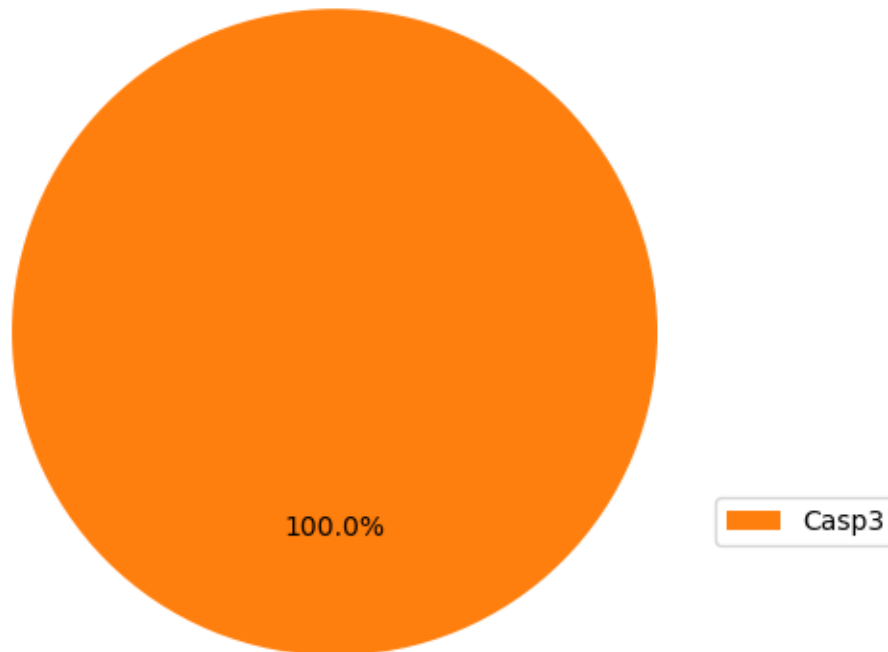
CPU times: user 1.72 ms, sys: 2.79 ms, total: 4.51 ms
Wall time: 10.4 s

```
[28]: # Plot the time plot and pie chart for Casp8 ectopic activity
mutres_Casp8_ON.get_nodes_probtraj().plot(legend=True)
plt.title(' Node trajectories for Casp8 ectopic activity')
mutres_Casp8_ON.plot_piechart()
plt.title('Repartition of final model states for Casp8 ectopic activity')
```

```
[28]: Text(0.5, 1.0, 'Repartition of final model states for Casp8 ectopic activity')
```



Repartition of final model states for Casp8 ectopic activity



Based on these results, we can conclude that the ectopic expression of Casp8 drives the cells into apoptosis faster, whether the death signal (TRAIL) is ON or OFF. This shows that death is not dependent on external signals in this context: the cell becomes independent from the death receptor engagement, and apoptosis is constantly activated.

3.8.2. Stochastic simulation of Casp8 knockdown We can also check the impact of a full knockdown of Casp8.

```
[29]: %%time
      # MaBoSS simulation for Casp8 knockdown
      mut_CASP8_OFF = WT.copy()
      mut_CASP8_OFF.mutate("Casp8", "OFF")
      mutres_CASP8_OFF = mut_CASP8_OFF.run()
```

CPU times: user 3.76 ms, sys: 2.16 ms, total: 5.93 ms
Wall time: 37 s

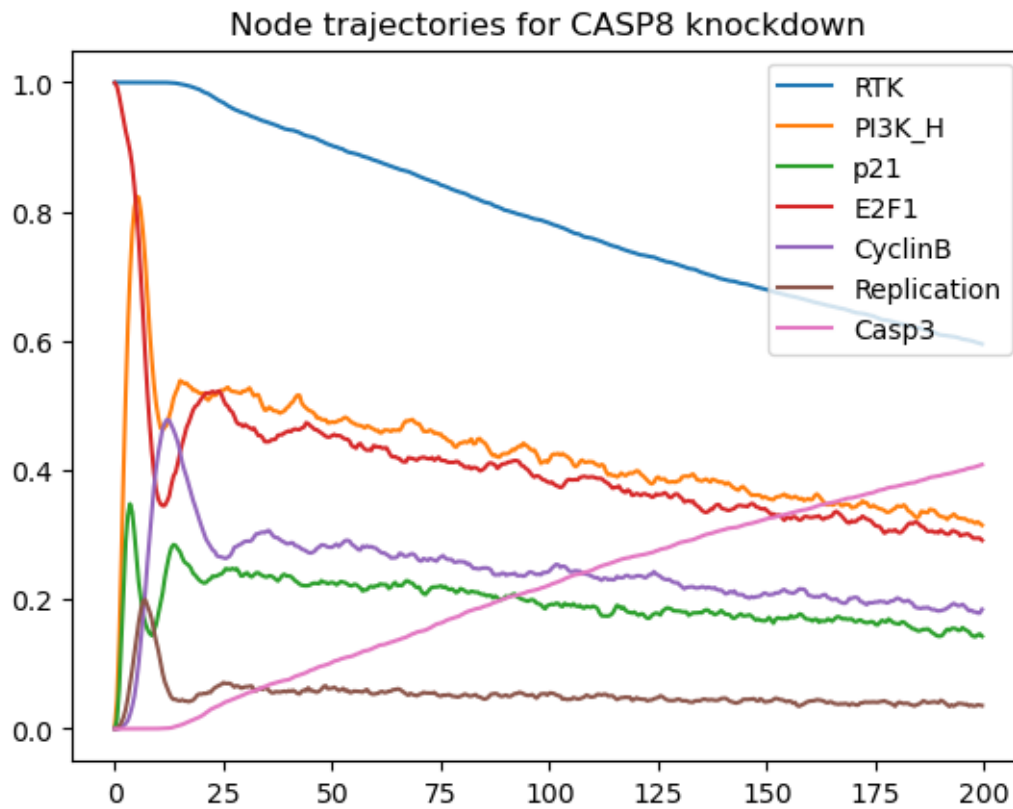
```
[30]: ## Plot the trajectories of the model states
      # mutres_CASP8_OFF.plot_trajectory(legend=True)
      # plt.title('Model state trajectories for CASP8 knockdown')

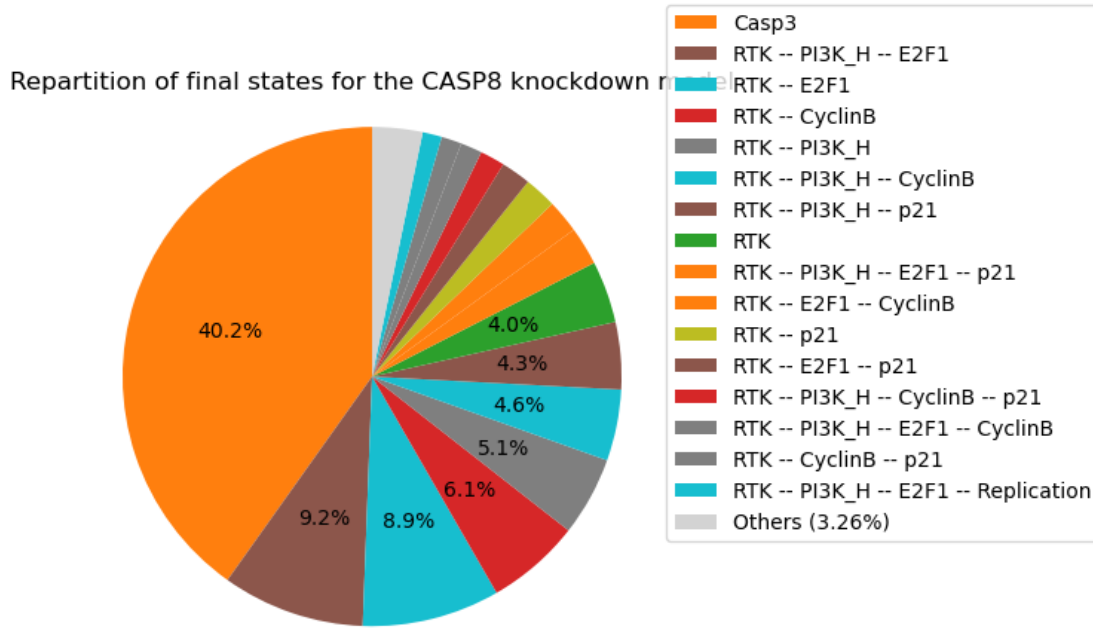
      # #Plot the time plot and pie chart for Casp8 ectopic activity
      mutres_CASP8_OFF.get_nodes_probtraj().plot(legend=True)
```



```
plt.title('Node trajectories for CASP8 knockdown')
mutres_CASP8_OFF.plot_piechart(legend=True)
plt.title('Repartition of final states for the CASP8 knockdown model')
```

[30]: Text(0.5, 1.0, 'Repartition of final states for the CASP8 knockdown model')





Casp3 is initially blocked but becomes ultimately activated, like in the wild-type case, independently of the status of Casp8. Hence, the model predicts that apoptosis (represented by Casp3 activity) can ultimately occur even in the absence of Casp8 activity. The knockout of Casp8 does not affect the model in the absence or presence of TRAIL.

Of note, it is possible to knock out a specific interaction with *bioLQM*, using the syntax: `target:regulator%0`, denoting a blockade of the interaction exerted by the regulator onto the target. Below, we provide the code to perform the simulation of such an edgetic perturbation blocking the regulation of Casp3 by Casp8.

```
[31]: m_perturbed = biolqm.perturbation(model_biolqm, "Casp3:Casp8%0")

# Translation of the edgetic variant model into MaBoSS format
perturbed_maboss = biolqm.to_maboss(model_biolqm)

# Specification of simulation parameters and selection of output nodes
perturbed_maboss.update_parameters(sample_count=5000,max_time=200, time_tick=0.
↪5)
perturbed_maboss.network.set_output(('CyclinB', 'FoxO3', 'Casp8', 'GSK3', 'BIM',
↪'BCL2', 'Casp3'))

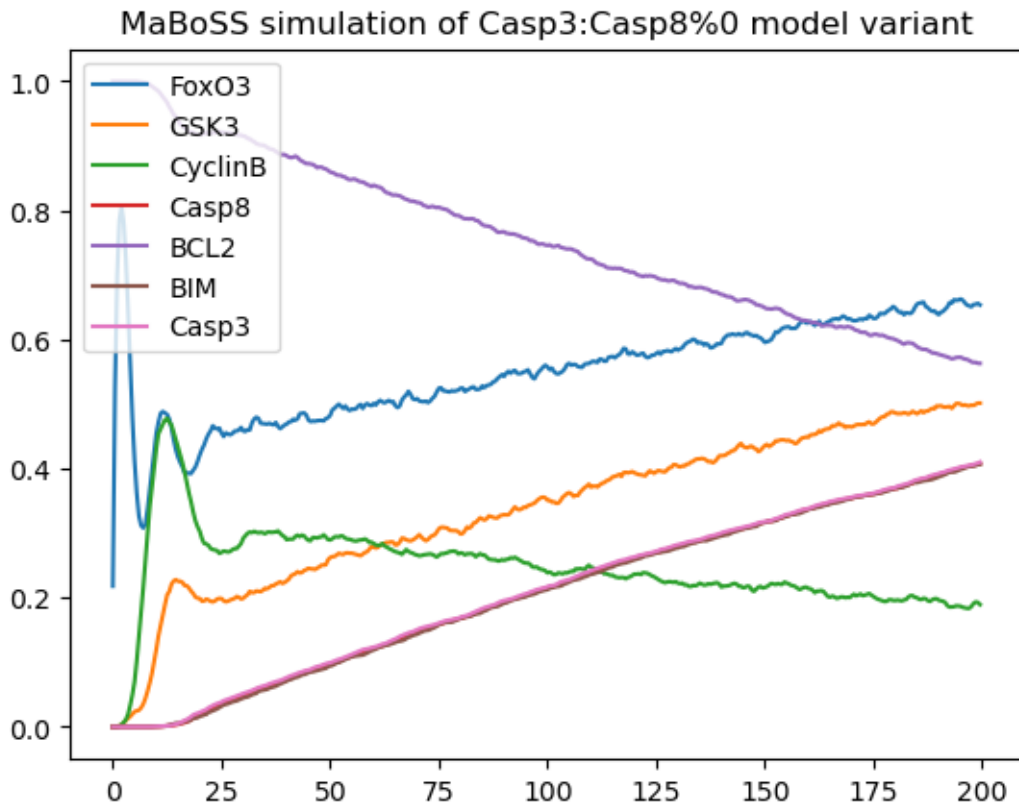
# Definition of the initial state, according to Fig. 5 of Sizek et al (2019).
active_init = [ k for (k,v) in init.items() if v==1 ]
maboss.set_nodes_istate(perturbed_maboss, active_init, [0,1])

# Run a MaBoSS simulation
```

```
run_perturbed_maboss = perturbed_maboss.run()

# Plot the trajectories of selected nodes
run_perturbed_maboss.get_nodes_probtraj().plot(legend=True)
plt.title('MaBoSS simulation of Casp3:Casp8%0 model variant')
```

```
[31]: Text(0.5, 1.0, 'MaBoSS simulation of Casp3:Casp8%0 model variant')
```



However, the simulation of this edgetic mutant does not bring any new insight into the behaviour of the model under consideration.

3.8.3. Multiple mutant simulations It is further possible to systematically explore a series of mutants, and to filter the results. In this respect, the notebook provides an example of such code, considering ten mutant simulations (inhibition and activation of five different nodes), and filtering those enabling a blockade of apoptosis, denoted by the suppression of Casp3 activation, while keeping the cell cycle active. This would correspond to a highly proliferative phenotype, with some alterations correlating with cancer patient conditions.

```
[32]: from maboss.pipelines import simulate_single_mutants, filter_sensitivity
list_mutants = ["PI3K_H", "p110_H", "p21", "Plk1", "FoxO3"]
explore_mutants = WT.copy()
```

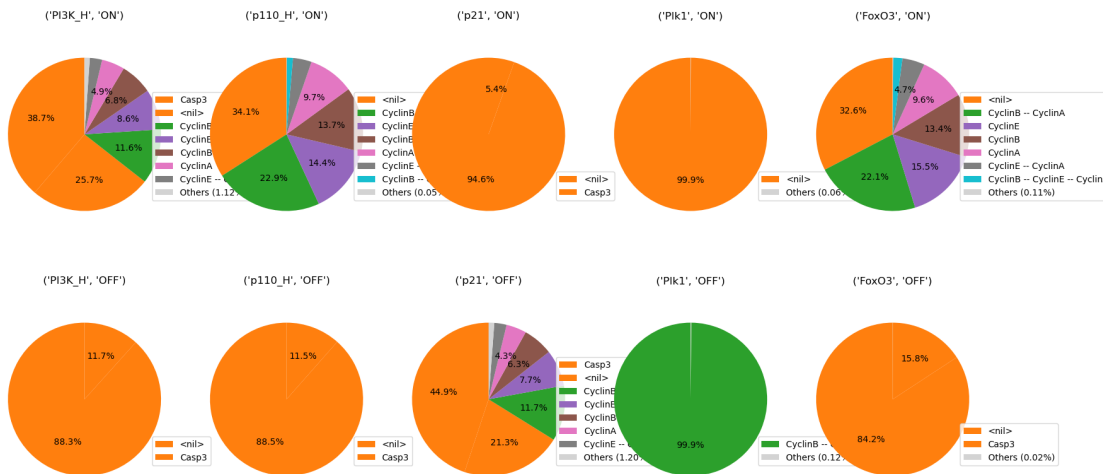
```
explore_mutants.network.set_output(["Casp3", "CyclinA", "CyclinB", "CyclinE"])
```

```
[33]: %%time
results = simulate_single_mutants(explore_mutants, list_mutants, sign='BOTH')
fig, ax = plt.subplots(2,5,figsize=(20,10))

for i, (name, result) in enumerate(results.items()):
    axes = ax[(i//5), (i%5)]
    result.plot_piechart(axes=axes)
    axes.set_title(name)
```

CPU times: user 214 ms, sys: 8.46 ms, total: 222 ms

Wall time: 3min 6s



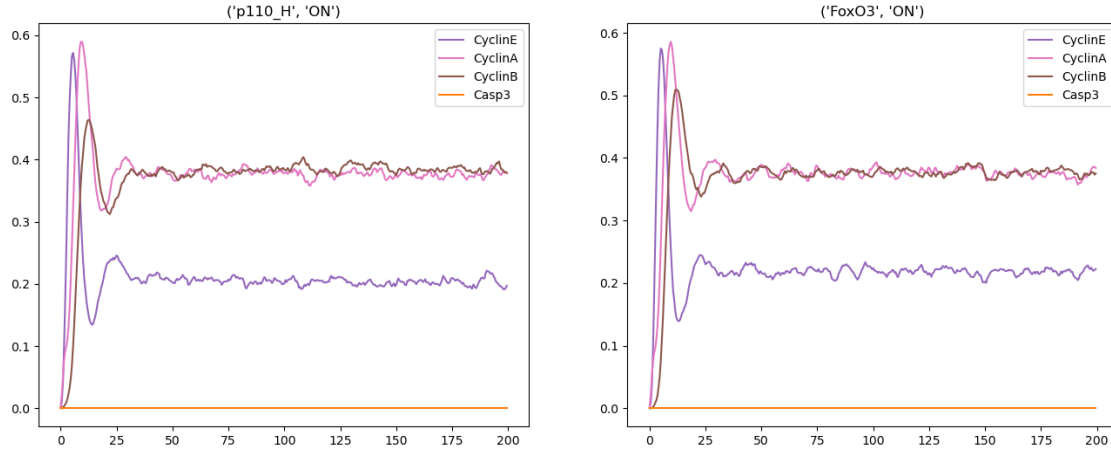
```
[34]: filtered_results = filter_sensitivity(results, node='Casp3', maximum=0.01)
filtered_results = filter_sensitivity(filtered_results, node='CyclinE',
    minimum=0.15)

fig, ax = plt.subplots(1,len(filtered_results),figsize=(16,6))

for i, (name, result) in enumerate(filtered_results.items()):
    axes = ax[i]
    result.plot_node_trajectory(axes=axes)
    axes.set_title(name)
```

/opt/conda/lib/python3.11/site-packages/maboss/figures.py:59: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

```
plt.legend(loc='upper right')
```



From this analysis, we can conclude that both p110 and FoxO3 sustained activations are able to turn off Casp3 activity, while maintaining the cycle. Note that we don't use a complete absence of Casp3 activity here (0%) as a filter but instead a quasi absence of activity (1%), as a very small activity might persist.

1.6.9 3.9. Deeper analysis of the quasi-cyclic synchronous attractor

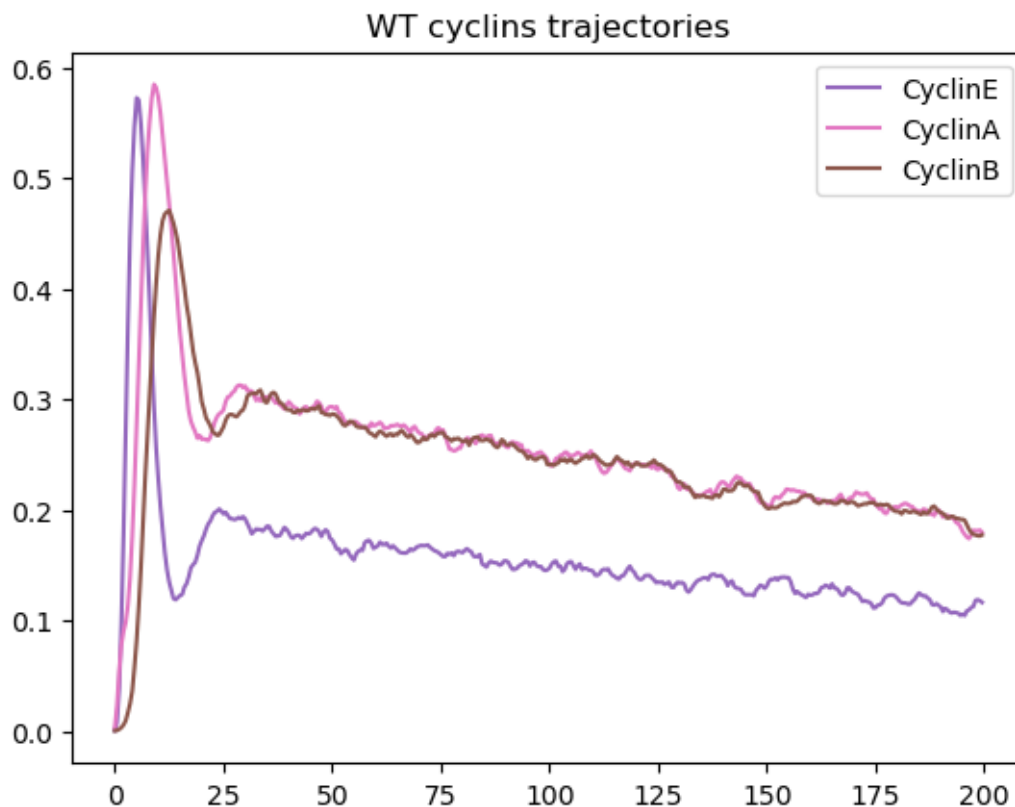
In this section, we explore the dynamics of the cell cycle by studying the succession of cyclin activations and inactivations. In continuous frameworks, i.e., models based on ordinary differential equations, it is possible to follow the activations of the cyclins that are crucial to ensure a proper cell cycle. In the WT situation, we expect that the cyclins get activated in the following order: CyclinE, followed by CyclinA, then CyclinB, and that they get inactivated in the same order. This sequence is not based on a simple cascade, but on complex regulation where new phases are controlled by checkpoints. In the case of stochastic Boolean simulations, the verification of this sequence of cyclin activations is more challenging. In this respect, we have developed a functionality reporting the probabilities of occurrences of specific node level transitions, thereby enabling the delineation of highly probable orders of activations of cyclins over time.

3.9.1. Verification of the order of activations of cyclins in the WT situation by plotting a compressed state transition graph The following code cells defines a simulation of the WT model focusing on cyclin dynamics.

```
[35]: outputs = ["CyclinA", "CyclinB", "CyclinE"]
      WT.network.set_output(outputs)
      %time res = WT.run()
      res.plot_node_trajectory()
      plt.title('WT cyclins trajectories')
```

CPU times: user 3.36 ms, sys: 54 s, total: 3.41 ms
Wall time: 30.9 s

```
[35]: Text(0.5, 1.0, 'WT cyclins trajectories')
```



We can see that the cyclins follow the expected order with the proper sequence of peaks: First CyclinE (red) around $t=5$, then CyclinA (purple) around $t=10$, and finally CyclinB (brown) around $t=15$. Note that as *MaBoSS* reports mean probabilities over time, these oscillations are rapidly damped.

We can further compute the frequencies of cyclin activations and inactivations over all individual trajectories of the simulation. To do this, we use a new functionality of *MaBoSS* estimating the frequencies of transitions between activity patterns involving selected nodes and plot them as a graph. To prevent the explosion of the size of this graph, we only use the three main cyclins responsible for the cyclic attractor. The following code cells compute such transition frequencies and display the results as a compressed state transition graph, i.e. a projection of the transition frequencies computed over the full state space onto the cyclin activity patterns of interest.

```
[36]: WT_trajs = WT.copy()
      WT_trajs.network.set_observed_graph_nodes(["CyclinA", "CyclinB", "CyclinE"])
```

```
[37]: %time res_trajs = WT_trajs.run()
```

CPU times: user 1.88 ms, sys: 986 s, total: 2.86 ms

Wall time: 32.5 s

```
[38]: res_trajs.get_observed_graph()
```

[38]:

	<nil>	CyclinB	CyclinE	\
State				
<nil>	0.000000	0.021081	0.800000	
CyclinB	0.980627	0.000000	0.000000	
CyclinE	0.130954	0.000000	0.000000	
CyclinB -- CyclinE	0.000000	0.588015	0.269663	
CyclinA	0.480899	0.000000	0.000000	
CyclinB -- CyclinA	0.000000	0.834242	0.000000	
CyclinE -- CyclinA	0.000000	0.000000	0.003002	
CyclinB -- CyclinE -- CyclinA	0.000000	0.000000	0.000000	

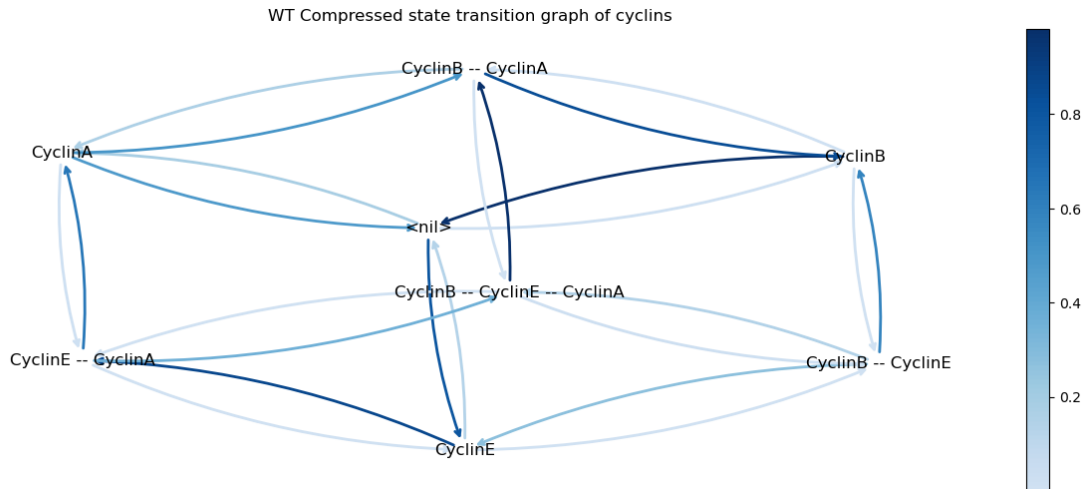
	CyclinB -- CyclinE	CyclinA	\
State			
<nil>	0.000000	0.178919	
CyclinB	0.006275	0.000000	
CyclinE	0.002041	0.000000	
CyclinB -- CyclinE	0.000000	0.000000	
CyclinA	0.000000	0.000000	
CyclinB -- CyclinA	0.000000	0.163611	
CyclinE -- CyclinA	0.000000	0.639002	
CyclinB -- CyclinE -- CyclinA	0.024134	0.000000	

	CyclinB -- CyclinA	CyclinE -- CyclinA	\
State			
<nil>	0.000000	0.000000	
CyclinB	0.013099	0.000000	
CyclinE	0.000000	0.867006	
CyclinB -- CyclinE	0.000000	0.000000	
CyclinA	0.504775	0.014326	
CyclinB -- CyclinA	0.000000	0.000000	
CyclinE -- CyclinA	0.000000	0.000000	
CyclinB -- CyclinE -- CyclinA	0.972931	0.002935	

	CyclinB -- CyclinE -- CyclinA
State	
<nil>	0.000000
CyclinB	0.000000
CyclinE	0.000000
CyclinB -- CyclinE	0.142322
CyclinA	0.000000
CyclinB -- CyclinA	0.002147
CyclinE -- CyclinA	0.357996
CyclinB -- CyclinE -- CyclinA	0.000000

```
[39]: fig, ax = plt.subplots(1,1,figsize=(16,6))
      res_trajs.plot_observed_graph(axes=ax)
      plt.title('WT Compressed state transition graph of cyclins')
```

```
[39]: Text(0.5, 1.0, 'WT Compressed state transition graph of cyclins')
```



In this graph, the transitions denoted by darker blue arrows have a higher probability of occurring. This representation enables us to visualise the most probable sequence of cyclin activations and inactivations. Starting from the state (corresponding to G0), the most probable sequence of state transitions successively involves CyclinE activation, CyclinA activation, CyclinE inactivation, CyclinB activation, CyclinA inactivation, and finally CyclinB inactivation, thereby completing the cycle. Note that other cycles can be followed with lower probabilities.

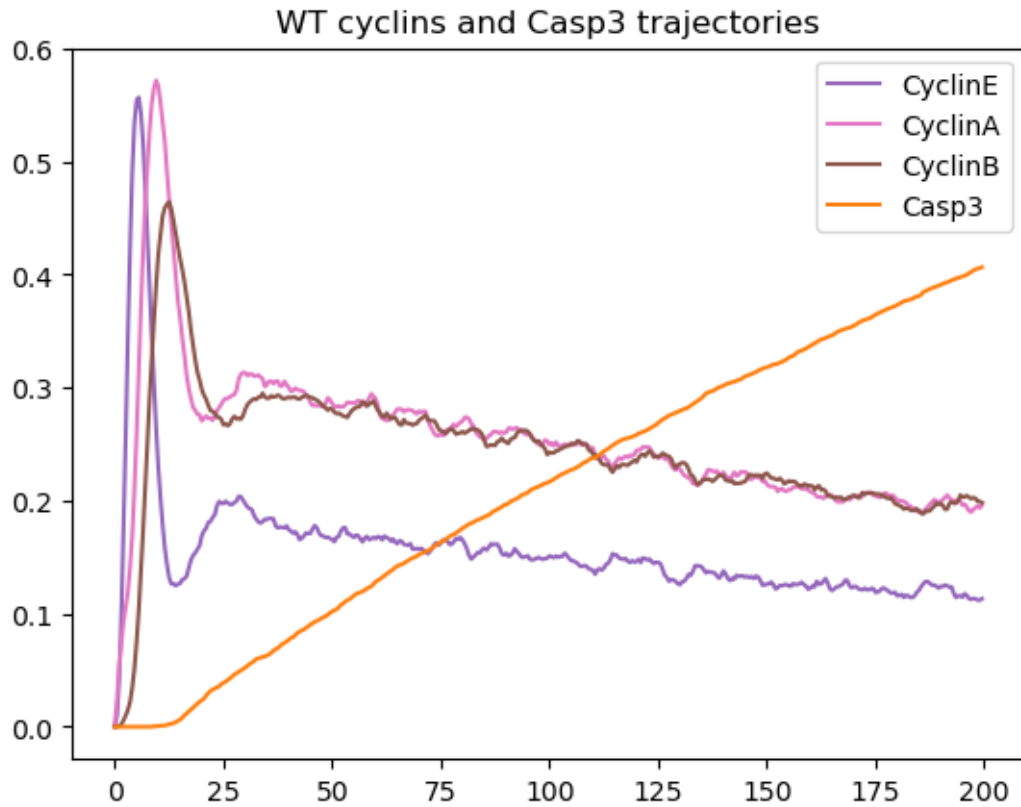
3.9.2. Analysis of the apoptosis triggering timing along the cell cycle Here we study the condition leading to apoptotic death, focusing on the point in the cell cycle where Casp3 is activated.

```
[40]: outputs = ["Casp3", "CyclinA", "CyclinB", "CyclinE"]
      WT.network.set_output(outputs)
      WT.network.set_observed_graph_nodes(outputs)
```

```
[41]: %time res = WT.run()
      res.plot_node_trajectory()
      plt.title('WT cyclins and Casp3 trajectories')
```

CPU times: user 2.38 ms, sys: 1.22 ms, total: 3.6 ms
Wall time: 31.8 s

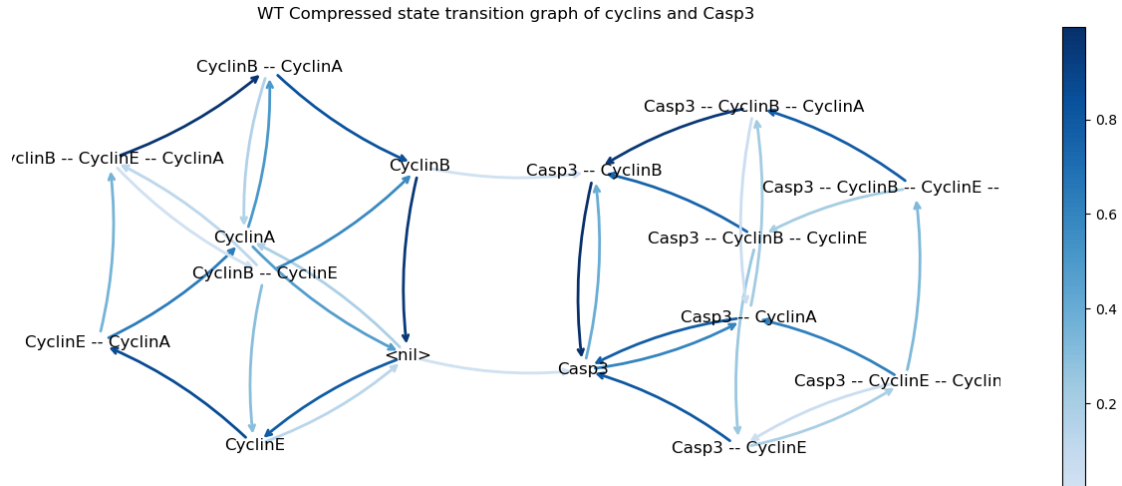
```
[41]: Text(0.5, 1.0, 'WT cyclins and Casp3 trajectories')
```

we can observe that cells initially follow the expected cycle, but quickly activate Casp3 and are thus led to apoptosis.

```
[42]: fig, ax = plt.subplots(1,1,figsize=(16,6))
      res.plot_observed_graph(axes=ax, prob_cutoff=0.02)
      plt.title('WT Compressed state transition graph of cyclins and Casp3')
```

```
[42]: Text(0.5, 1.0, 'WT Compressed state transition graph of cyclins and Casp3')
```



Note that we used a minimum probability threshold (2%) to discard unlikely edges during plotting. We see that there are two main ways to activate apoptosis: from $\langle \text{nil} \rangle$, corresponding to G0, and when cyclin B is active, denoting the G2/M phase.

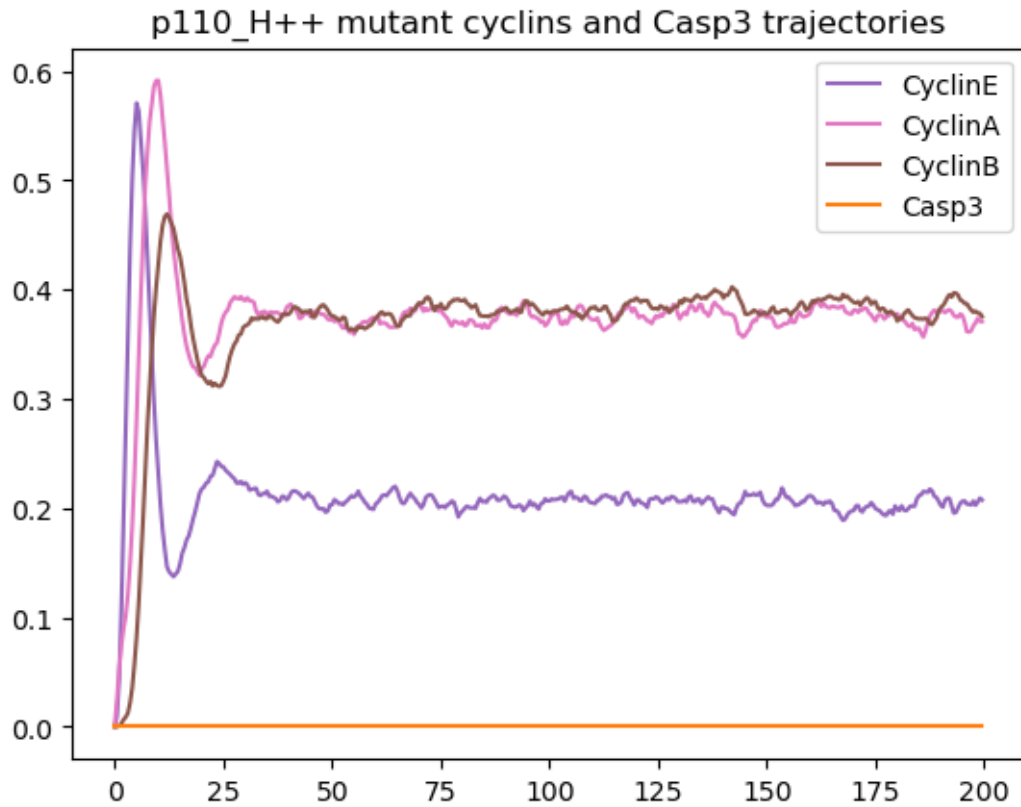
3.9.3. Analysis of cyclin triggering timing in presence of ectopic p110_H activity In the initial publication, p110 activates both PI3K and AKT, which in turn inhibit cell death. In the mutant analyses performed by the authors, p110 played a key role in the interaction between the MAPK module and the apoptotic module. To verify the impact of a forced p110_H activation, we can execute the following code cell:

```
[43]: # Copy the WT model and block p110_H ON
mut_p110 = WT.copy()
mut_p110.mutate('p110_H', 'ON')
```

```
[44]: # Simulation of the mutant model
%time res_mut_p110 = mut_p110.run()
res_mut_p110.plot_node_trajectory()
plt.title('p110_H++ mutant cyclins and Casp3 trajectories')
```

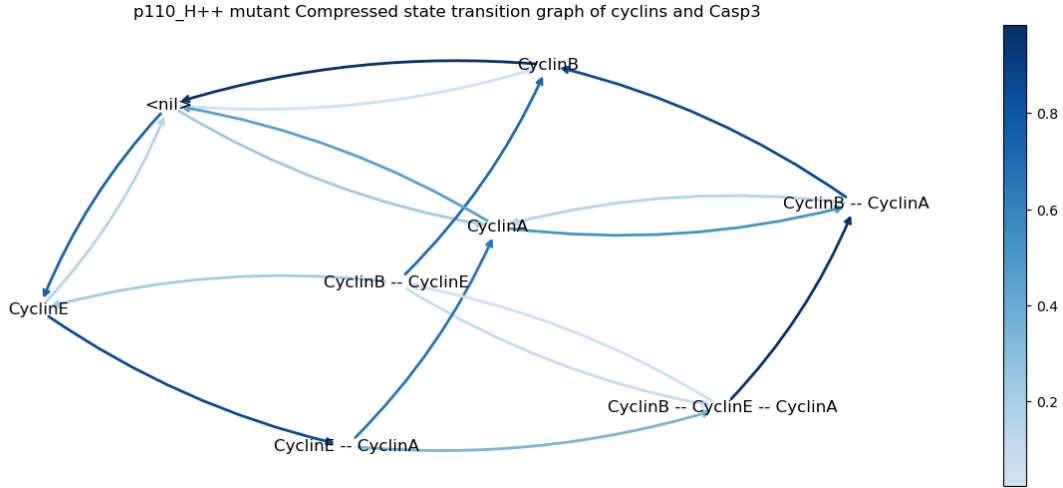
CPU times: user 432 s, sys: 2.08 ms, total: 2.51 ms
Wall time: 21.7 s

```
[44]: Text(0.5, 1.0, 'p110_H++ mutant cyclins and Casp3 trajectories')
```



```
[45]: # Plot the compressed transition graph
fig, ax = plt.subplots(1,1,figsize=(16,6))
res_mut_p110.plot_observed_graph(axes=ax, prob_cutoff=0.02)
plt.title('p110_H++ mutant Compressed state transition graph of cyclins and
↪Casp3')
```

```
[45]: Text(0.5, 1.0, 'p110_H++ mutant Compressed state transition graph of cyclins and
Casp3')
```



As can be seen in the resulting graph, the ectopic activity of p110_H completely disables the activation of apoptosis, enabling the maintenance of the oscillatory behaviour.

1.7 4. Conclusion

After providing the instructions to install the 20 different tools integrated in the *CoLoMoTo* software suite, this tutorial focused on the analysis of a published model of the regulatory network controlling mammalian cell proliferation [13], which can be further used as a template for the analysis of any other Boolean model in standard format.

The tutorial includes python code enabling the reproduction of several of the results reported by the initial publication [13], as well as to further extend these results with new analyses. Using selected tools included in the *CoLoMoTo* suite, we showed how to compute the model attractors with synchronous or asynchronous updating. Although stable states are naturally preserved, our analysis emphasises the sensitivity of synchronous cyclic attractors to the update mode.

We further showed how the use of a probabilistic simulation framework, implemented in the *MaBoSS* software, can provide interesting information on the transient behaviour of such a complex model, for wild-type conditions, as well as for different types of model perturbations. Integrating all these analyses into an executable Jupyter notebook greatly facilitates their reproducibility, as well as extensions. The notebook can also be used as a template for encoding completely new model analyses. Due to space constraints, this notebook explicitly covers the use of only a fraction of the tools available in the *CoLoMoTo* collection. However, all integrated tools are accompanied by short specific tutorials available in the *CoLoMoTo Docker* container, as well as on the *CoLoMoTo* website. The analysis presented in this tutorial can be easily modified or extended by taking advantage of other tools included in the *CoLoMoTo* environment. For example, simulations with different updating modes could be performed with *mpbn* or *BooleanNet*; *BooN* could be used for stable state analysis; *Pint* could be used to perform automatic search of mutations enforcing or impeding the reachability of a specific attractor; models could be import from other databases, such as *Cell Collective*, in different formats, such as *BoolSim* or *BoolNet* formats.

Acknowledgements We are grateful to Claudine Chaouiya and Pedro Monteiro for their key contri-

butions to the development of GINsim and to our many colleagues who contributed tools integrated in the CoLoMoTo environment.

Data accessibility.

The notebook and required companion files are available at the url: <https://github.com/colomoto/colomoto-tutorial-sizek>.

The source notebook file (with a .ipynb extension) can be uploaded and executed within the Jupyter interface of the CoLoMoTo notebook, using the Docker image `colomoto/colomoto-docker:2025-03-01`.

Competing interests.

We declare we have no competing interests.

Funding.

This work has been supported by the French Plan Cancer-MIC ITMO, Project *MoDICEd* (No. 20CM114-00), by the French Agence Nationale pour la Recherche (ANR) in the scope of the project *BNeDiction* (grant number ANR-20-CE45-0001), and by the France 2030 project *AI4scMED* operated by ANR (grant number ANR-22-PESN-0002).

1.8 References

1. Sugita M. Functional analysis of chemical systems in vivo using a logical circuit equivalent. II. The idea of a molecular automaton. *J Theor Biol* 1963; 9-184. ([doi:10.1016/0022-5193\(63\)90027-4](https://doi.org/10.1016/0022-5193(63)90027-4))
2. Kauffman SA. Metabolic stability and epigenesis in randomly constructed genetic nets. *J Theor Biol* 1969; 22: 437-467. ([doi:10.1016/0022-5193\(69\)90015-0](https://doi.org/10.1016/0022-5193(69)90015-0))
3. Thomas R. Boolean formalization of genetic control circuits. *J Theor Biol* 1973; 42: 563-585. ([doi:10.1016/0022-5193\(73\)90247-6](https://doi.org/10.1016/0022-5193(73)90247-6))
4. Albert R, Thakar J. Boolean modeling: a logic-based dynamic approach for understanding signaling and regulatory networks and for making useful predictions. *Syst Biol Med* 2014; 6: 353-369. ([doi:10.1002/wsbm.1273](https://doi.org/10.1002/wsbm.1273))
5. Abou-Jaoudé W, Traynard P, Monteiro P, Saez-Rodriguez J, Helikar T, Thieffry D, Chaouiya D. Logical modeling and dynamical analysis of cellular networks. *Front Genet* 2016; 7: 94. ([doi:10.3389/fgene.2016.00094](https://doi.org/10.3389/fgene.2016.00094))
6. Schwab JD, Kühlwein SD, Ikononi N, Kühl M, Kestler HA. Concepts in Boolean network modeling: What do they all mean? *Comput Struct Biotechnol J*. 2020; 18 :582 ([doi:10.1016/j.csbj.2020.03.001](https://doi.org/10.1016/j.csbj.2020.03.001))
7. Thakar J. Pillars of biology: Boolean modeling of gene-regulatory networks. *J Theor Biol* 2024; 578: 111682. ([doi:10.1016/j.jtbi.2023.111682](https://doi.org/10.1016/j.jtbi.2023.111682))
8. Zerrouk N, Augé F, Niarakis A. Building a modular and multi-cellular virtual twin of the synovial joint in Rheumatoid Arthritis. *NPJ Digit Med* 2024; 7: ([doi:10.1038/s41746-024-01396-y](https://doi.org/10.1038/s41746-024-01396-y))
9. Hemedan AA, Niarakis A, Schneider R, Ostaszewski M. Boolean modelling as a logic-based dynamic approach in systems medicine. *Comput Struct Biotechnol J* 20: 3161-3172. ([doi:10.1016/j.csbj.2022.06.035](https://doi.org/10.1016/j.csbj.2022.06.035))
10. Tiwari K, Kananathan S, Roberts MG, Meyer JP, Shohan MUS, Xavier A, Maire M, Zyoud A, Men J, Ng S, Nguyen TVN, Glont M, Hermjako, Malik-Sheriff RS. Reproducibility in systems biology modelling. *Mol Syst Biol* 2021; 17: e9982. ([doi: 10.15252/msb.20209982](https://doi.org/10.15252/msb.20209982))

11. Chaouiya C, Berenguier D, Keating SK, Naldi A, van Iersel MP, Rodriguez N, Dräger A, Büchel F, Cokelaer T, Kowal B, Wicks B, Gonçalves E, Dorier J, Page onteiro PT, von Kamp A, Xenarios I, de Jong H, Hucka M, Klamt S, Thieffry D, Le Novère N, Saez-Rodriguez J, Helikar T. SBML qualitative models: a model repr ation format and infrastructure to foster interactions between qualitative modelling formalisms and tools. *BMC Syst Biol* 2013; 7: 135. ([doi:10.1186/1752-05135](https://doi.org/10.1186/1752-05135))
12. Naldi A, Hernandez C, Levy N, Stoll G, Monteiro PT, Chaouiya C, Helikar T, Zinovyev A, Calzone L, Cohen-Boulakia S, Thieffry D, Paulevé L. The CoLoMoTo active Notebook: Accessible and Reproducible Computational Analyses for Qualitative Biological Networks. *Front Physiol* 2018; 9: 680. ([doi:10.3389/fphys.2018.00680](https://doi.org/10.3389/fphys.2018.00680))
13. Sizek H , Hamel A , Deritei D, Campbell S , Regan ER. Boolean model of growth signaling, cell cycle and apoptosis predicts the molecular mechanism of aberrant cell cycle progression driven by hyperactive PI3K. *PLoS Comput Biol* 2019; 15: e1006402. ([doi:10.1371/journal.pcbi.1006402](https://doi.org/10.1371/journal.pcbi.1006402))
14. Naldi A, Hernandez C, Abou-Jaoudé W, Monteiro PT, Chaouiya C, Thieffry D. Logical modelling and analysis of cellular regulatory networks with GINsim 3.0. *Front Physiol* 2018; 9: 646. ([doi:10.3389/fphys.2018.00646](https://doi.org/10.3389/fphys.2018.00646))
15. Naldi A. BioLQM: A Java Toolkit for the Manipulation and Conversion of Logical Qualitative Models of Biological Networks. *Front Physiol* 2018; 9: 1605. ([doi:10.3389/fphys.2018.01605](https://doi.org/10.3389/fphys.2018.01605))
16. Naldi A, Remy E, Thieffry D, Chaouiya C. Dynamically consistent reduction of logical regulatory graphs. *Theor Comput Sci* 2011; 412: 2207-2218. ([doi:10.1016/j.tcs.2010.10.021](https://doi.org/10.1016/j.tcs.2010.10.021))
17. Stoll G, Caron B, Viara E, Dugourd A, Zinovyev A, Naldi A, Kroemer G, Barillot E, Calzone L. MaBoSS 2.0: an environment for stochastic Boolean modeling. *Bioinformatics* 2017; 33: 2226-2228. ([doi:10.1093/bioinformatics/btx123](https://doi.org/10.1093/bioinformatics/btx123)).
18. Stoll G., Viara E., Barillot E., Calzone L. Continuous time boolean modeling for biological signaling: application of Gillespie algorithm. *BMC Syst Biol* 2012, 6: 116. ([doi:10.1186/1752-0509-6-116](https://doi.org/10.1186/1752-0509-6-116)).

[]: